

Design Review — RDM

Document ID	DR-001
Revision	81
Date	2026-10-04
Status	Controlled in git

Contents

1 Purpose	6
2 Design Review 1 — Design inputs and architecture	6
2.1 Participants	6
2.2 Items reviewed	6
2.3 Test-faithfulness review (per design input)	6
2.4 Findings and actions	7
3 Design Review 2 — Agent enablement and document control	7
3.1 Items reviewed	7
3.2 Findings and actions	7
4 Design Review 3 — Soundness hardening, release artifacts, audit response	8
4.1 Items reviewed	8
4.2 Findings and actions	8
5 Design Review 4 — Retiring the faithfulness gate	8
5.1 Items reviewed	9
5.2 Findings and actions	9
6 Design Review 5 — Design record as a linked-data graph	9
6.1 Items reviewed	9
6.2 Findings and actions	10
7 Design Review 6 — Product structure	10
7.1 Items reviewed	10
7.2 Findings and actions	10
8 Design Review 7 — Measurement faults	10
8.1 Items reviewed	11
8.2 Findings and actions	11
9 Design Review 8 — Read-only agent interface	11
9.1 Items reviewed	11
9.2 Findings and actions	11
10 Design Review 9 — Risk register	12
10.1 Items reviewed	12
10.2 Findings and actions	12
11 Design Review 10 — Risk evaluation per the requirements skill	12
11.1 Items reviewed	13
11.2 Findings and actions	13
12 Design Review 11 — One data model: the record and its graph	13
12.1 Items reviewed	13
12.2 Findings and actions	14
13 Design Review 12 — Findings from the graph's own analysis	14
13.1 Items reviewed	14
13.2 Findings and actions	14
14 Design Review 13 — Test evidence	14
14.1 Items reviewed	15
14.2 Findings and actions	15

15	Design Review 14 — Allure results as RDF	15
15.1	Items reviewed	15
15.2	Findings and actions	15
16	Design Review 15 — The Allure behaviors hierarchy	16
16.1	Items reviewed	16
16.2	Findings and actions	16
17	Design Review 16 — No island documents, second pass	16
17.1	Items reviewed	17
17.2	Findings and actions	17
18	Design Review 17 — Edges that claim nothing	17
18.1	Items reviewed	17
18.2	Findings and actions	17
19	Design Review 18 — Noise out of the graph	18
19.1	Items reviewed	18
19.2	Findings and actions	18
20	Design Review 19 — Evidence that says what it is evidence for	18
20.1	Items reviewed	18
20.2	Findings and actions	19
21	Design Review 20 — Rules for what the graph derives	19
21.1	Items reviewed	19
21.2	Findings and actions	19
22	Design Review 21 — One home for each rule and helper	20
22.1	Items reviewed	20
22.2	Findings and actions	20
23	Design Review 22 — The gates as reusable CI	20
23.1	Items reviewed	20
23.2	Findings and actions	21
24	Design Review 23 — The verification report	21
24.1	Items reviewed	21
24.2	Findings and actions	21
25	Design Review 24 — The verification report, for an auditor	21
25.1	Items reviewed	22
25.2	Findings and actions	22
26	Design Review 25 — One vocabulary with the requirements skill	22
26.1	Items reviewed	22
26.2	Findings and actions	23
27	Design Review 26 — Verification steps are not acceptance criteria	23
27.1	Items reviewed	23
27.2	Findings and actions	23
28	Design Review 27 — The architecture as C4, checked against the code	23
28.1	Items reviewed	24
28.2	Findings and actions	24
29	Design Review 28 — Mermaid diagrams in rendered documents	24
29.1	Items reviewed	24
29.2	Findings and actions	25
30	Design Review 29 — The architecture as one workspace, its views as images	25

30.1 Items reviewed	25
30.2 Findings and actions	26
31 Design Review 30 — Bounded contexts drawn by language	26
31.1 Items reviewed	26
31.2 Findings and actions	27
32 Design Review 31 — Every design document to one template	27
32.1 Items reviewed	27
32.2 Findings and actions	28
33 Design Review 32 — One package per context, and the rule enforced	28
33.1 Items reviewed	28
33.2 Findings and actions	29
34 Design Review 33 — A design input is stated once	29
34.1 Items reviewed	29
34.2 Findings and actions	29
35 Design Review 34 — The design outputs are the architecture	30
35.1 Items reviewed	30
35.2 Findings and actions	30
36 Design Review 35 — Commands and events: the domain model	30
36.1 Items reviewed	31
36.2 Findings and actions	31
37 Design Review 36 — Commands and events, trimmed	31
38 Design Review 37 — The release gate names its events	31
38.1 Items reviewed	32
39 Design Review 38 — The design gate names its events	32
40 Design Review 39 — The risk rules name their events	32
41 Design Review 40 — The C4 model checked against the record; components in the trace .	33
42 Design Review 41 — What the C4 shapes found in RDM's own record	33
43 Design Review 42 — The evidence bundle keeps the run's executor and environment	34
44 Design Review 43 — RDM 2.0.0-alpha, and the version adoption pins	34
45 Design Review 44 — What cannot be read fails the gate	34
46 Design Review 45 — The risk rules over what a register really says	35
47 Design Review 46 — Nothing served reaches the network	35
48 Design Review 47 — Evidence that says what happened	36
49 Design Review 48 — The rest of what the exploratory test found	37
50 Design Review 49 — A run finds its test in a nested project	37
51 Design Review 50 — The record says what it holds, or the gate fails	38
52 Design Review 51 — A risk rating no one approved is never read as approved	38
53 Design Review 52 — Every reader of the results agrees with the release gate	39
54 Design Review 53 — The graph blocks what the gates block	40
55 Design Review 54 — The hook gates every way implementation is committed	40
56 Design Review 55 — Gap analysis and publishing say what is wrong	41
57 Design Review 56 — Starting a project, adopting one and adding an input work as the docs say	42
58 Design Review 57 — Review findings on the behaviour fixes	43
59 Design Review 58 — Design input by design input, the whole chain	43
59.1 DI-1 — Reading the record, with no planning dependency	43

59.2	DI-2 — The design gate blocks implementation until the design is approved	44
59.3	DI-3 — Release is blocked until every design input is verified	44
59.4	DI-4 — Results reconcile to a status per design input and render as a matrix	45
59.5	Approved changes	45
60	Design Review 59 — Why tests drifted from their design inputs, and the process that holds them	46
60.1	What happened	46
60.2	Why	46
60.3	The answer is a process, not automation	47
61	Design Review 60 — Design input by design input, continued	47
61.1	DI-5 — Persona runs give a formative status per user need	47
61.2	DI-7 — A Markdown template rendered against the data supplied	48
61.3	DI-8 — The traceability template filters	48
61.4	DI-9 to DI-70 — the rest of the pass	49
62	Design Review 61 — What a test exercises, without reading the code	52
62.1	Why	52
62.2	Proposed	53
62.3	Decisions	53
62.4	Approved changes	53
63	Design Review 62 — RDM's own runs attach their coverage	54
64	Design Review 63 — Coverage is a unit-test measure, never acceptance evidence	54
65	Design Review 64 — The unit tests' code coverage, as evidence of unit verification	55
66	Design Review 65 — The map draws what the graph holds	55
67	Design Review 66 — The design outputs follow the behaviour of Design Reviews 58 to 65	56
68	Design Review 67 — The user manual is a design output, held in the DMR and the graph ..	56
69	Design Review 68 — A blind provenance check, from the graph alone	57
70	Design Review 69 — The instructions for use as one document	58
71	Design Review 70 — The IFU in a developer's words	58
72	Design Review 71 — One copy of the docs per release	59
73	Approval	59

Id: DR-001 · Revision: 81

1 Purpose

Records the design review of RDM's design inputs (DI-001) and the record-first architecture.

2 Design Review 1 – Design inputs and architecture

Scope reviewed: Design inputs (DI-001) and the system architecture (`architecture.md`), against the record-first model in `docs/record-first-architecture.md` and ADR 0001.

Disposition: Approved.

2.1 Participants

Recorded via the version-control history: the reviewers are the approvers of the pull request in which this review was merged. At least one approver is independent of the authoring of the reviewed design stage (approver ≠ commit author).

2.2 Items reviewed

- Design inputs DI-1..DI-6 are unambiguous and individually verifiable by an automated test.
- Each user need (V&V plan) is addressed by at least one bounded-context design document via `satisfies`; no user need is duplicated across contexts.
- The record core (record, gating, verification, validation, rendering) does not depend on the planning extra; planning is fenced as non-record (DI-6).
- Approval is the version-control record; no duplicate sign-off is introduced.
- Verification (Allure) and validation (human summative + persona formative) are distinguished; persona evidence never gates release.

2.3 Test-faithfulness review (per design input)

The detailed §820.30(e) examination of *whether each verifying test actually verifies its design input* is recorded as machine-checkable, hash-pinned verdicts under `dhf/faithfulness/` (`rdm story faithfulness` reconciles them). At this revision every design input (DI-1..DI-6) carries a current `faithful` verdict; the release gate blocks if any is missing, negative, or stale (test changed since review). One weak test was found during this review (DI-6 originally asserted only the wording of `PROVENANCE_NOTE`) and was strengthened to assert the actual stamping behaviour before being marked faithful.

2.4 Findings and actions

- No blocking findings. The release gate and traceability still read the user-need registry from per-context SDD frontmatter in code; migrating them to read the V&V-plan registry (ADR 0001 consequence) is tracked as follow-up and does not block this review.

3 Design Review 2 – Agent enablement and document control

Scope reviewed: the design inputs added for agent-era contributor enablement and document control — DI-22 (design-input scaffolding, `rdm_story_new-input`), DI-23 (record-first-aware traceability audit), DI-24 (brownfield adoption, `rdm adopt`), and DI-25 (Part 11 document-control checklist + RDM's own document-control statement, `document_control.md`) — together with the user needs they refine (UN-010, UN-011) and the canonical change procedure (`AGENT_WORKFLOW.md`).

Disposition: Approved.

3.1 Items reviewed

- Each new design input is unambiguous, owned by exactly one context (scaffolding, `story_audit`, `gap_analysis`), and individually verified by an `@allure.story`-tagged acceptance test.
- Every user need in the V&V plan registry is addressed by at least one design input; the release gate enforces the full denominator.
- The document-control statement's Part 11 mapping is held to the shipped `part11_document_control` checklist by an executable, falsifiable check.
- Independence of the §820.30(e) faithfulness review was exercised, not just asserted: reviews for DI-22 and for the worked example's DI-6..8 initially returned `partial` with executed surviving mutations; the verifying tests were strengthened and re-reviewed to `faithful`. The partial verdicts and the strengthening commits remain in history as the audit trail of the loop working.
- The pre-commit design gate blocked an implementation commit staged ahead of its design approval during this cycle (DI-25) — the enforced sequencing operates as designed.

3.2 Findings and actions

- No blocking findings. The follow-up from Review 1 (migrating the release gate to read the V&V-plan registry) is implemented; the release-gate denominator is the union of `design_inputs` reconciled against the registry.

4 Design Review 3 – Soundness hardening, release artifacts, audit response

Scope reviewed: the design inputs added or amended after Review 2 — DI-26 (design-gate-only hooks default), DI-27 (replayable probes, report filters), DI-28 (per-verdict hash scope), DI-29/DI-30 (DMR index data and the release evidence bundle, with UN-012), DI-31 (polyglot tag discovery), DI-32 (legacy-workflow deprecation), the DI-21 amendment (defense-in-depth probe restore after the SIGTERM/stale-pyc incidents), the DI-10/DI-22 amendments (sound gap reference matching; satisfies-list sync), and DI-33 (summative validation records + release-gate warnings, raised by external-style audit finding NC-1).

Disposition: Approved.

4.1 Items reviewed

- Each input is owned by one context, individually verified by a tagged test, and carries a current independent faithfulness verdict; from DI-26 onward verdicts embed executed mutation probes and are replay-verified.
- The incident record (SIGTERM restore gap; same-second stale-pyc; the replay timeout) shows root-cause fixes flowing through this same loop, with the interim partial verdicts retained as evidence.
- Audit findings NC-4 (checklist completeness: §11.10(h)/(j)), NC-5 (this review entry), and NC-6 (retention statement) are dispositioned in this revision; NC-1 is dispositioned by DI-33 plus the summative reviews that remain a human obligation; NC-2/NC-3 (human PR review and approval; unique, signed identities) are open human actions tracked in the pull request.

4.2 Findings and actions

- Open: summative validation records for UN-001..012 (human reviewers, per the V&V plan approach table) — the release gate now names each missing record.
- Open: merge via an independent, human-reviewed PR; individual signing identities (NC-2/NC-3).

5 Design Review 4 – Retiring the faithfulness gate

Scope reviewed: removal of the per-design-input faithfulness review — DI-19 (faithfulness gate), DI-20 (verdict recorder), DI-21 (mutation probe), DI-27 (replayable probes), DI-28 (verdict hash scope) and the user need they refined, UN-009 — plus the wording amendments that follow from it: DI-30 (the evidence bundle no longer carries verdicts), DI-31 (tag discovery no longer feeds verdict hashing), UN-012, and the document-control statement.

Disposition: Approved.

5.1 Items reviewed

- Rationale: the faithfulness verdicts were RDM's own construct, not a §820.30 or IEC 62304 requirement. Their hash pins re-opened reviews on edits that changed no requirement and no product behavior (a one-line test edit, a shared test module, a dependency declaration), and the defects they found were overwhelmingly in RDM's own verification machinery. An experiment (branch `claude/phoenix-borrowings`, not merged) added six more mechanisms to make that cost bearable, which confirmed the direction was wrong.
- Independent verification is now the human-reviewed pull request: git is the controlled record and audit trail, the ruleset requires an approving review by someone other than the author, and CI runs design-gate → acceptance tests → verify → release-gate on every change. A passing tagged test per design input remains the release condition (DI-3).
- Retired ids (DI-19, DI-20, DI-21, DI-27, DI-28, UN-009) are not reused. Their history, verdicts and probes remain in git.

5.2 Findings and actions

- Accepted: a test can pass without proving its requirement; detecting that is the pull-request reviewer's job. Teams that want automated evidence can run a mutation-testing tool in CI.
- Addendum: the mutation probe is kept as a standalone reviewer tool (DI-34, refining the new UN-013), so that judgment can be backed by an executed check. It records no verdict and gates nothing; DI-21 stays retired.
- Open (carried from Review 3): summative validation records; human PR review and individual signing identities (NC-2/NC-3).

6 Design Review 5 – Design record as a linked-data graph

Scope reviewed: UN-014, the new `graph` context, DI-35 (RDF projection of the record) and DI-36 (Oxigraph store, SPARQL query, SPARQL endpoint for AWS Graph Explorer).

Disposition: Approved.

6.1 Items reviewed

- Consistent with RDM-004's settled rule that Markdown is the authoring surface and any database is a derived projection: the graph is rebuilt from the record, never edited, and Graph Explorer is a browser, not an editor.
- Open-world semantics are explicit: the graph asserts only recorded facts; pass/fail judgments stay in the gates (a later SHACL step may express them as shapes — not in scope here).
- Store choice: Oxigraph (embedded, SPARQL 1.1, named graphs, persistent, actively released); Kuzu ruled out (property graph, archived Oct 2025); Jena/GraphDB/RDFox deferred (server, proprietary, or licensed).
- Optional extra: the record core keeps no RDF dependency.

6.2 Findings and actions

- Open: whether this projection is the answer to RDM-004.06 (types as data) is decided on that ticket, not here.
- Not in scope: backlog/plan graph, OWL reasoning.
- Addendum: DI-37 adds the gap-analysis half of the record (checklist clauses and [...] reference tags, matched by `rdm gap`'s own code) and DI-38 the gate rules as SHACL shapes (pySHACL), held by test to agreement with the release gate and `rdm gap`. The coded gates remain authoritative.

7 Design Review 6 – Product structure

Scope reviewed: the intended use (V&V plan) and the system architecture (`architecture.md`), restated as four parts — Record, Gates, Graph, Documents — with the ten existing contexts assigned to them.

Disposition: Approved.

7.1 Items reviewed

- No design input, user need or module changes; this is the record of how the existing contexts fit together.
- The record is the only thing people and agents write, and only through a reviewed pull request. Graph, matrix, documents and evidence are derived and never edited — consistent with Design Review 5 and RDM-004.
- The graph is the read interface for agents; it accepts no writes.
- Agent skills are maintained outside RDM (`scope-impact/agent-skills`).

7.2 Findings and actions

- Open: risk management, regulation-to-checklist, and a read-only agent interface are named as not yet in scope; each enters as a user need with design inputs.
- Open: known measurement faults in the record — duplicate Allure results from the docs build, test tags read from string literals, and the Part 11 document-control claims not linked to any design input.

8 Design Review 7 – Measurement faults

Scope reviewed: DI-40 (record context) and the three measurement faults left open by Design Review 6.

Disposition: Approved.

8.1 Items reviewed

- Test tags read from string literals: a product defect in Python tag discovery. DI-40 reads tags from the syntax tree instead.
- Duplicate Allure results: the docs build and local runs append to `dhf/allure-results`, so a second run doubles every result. Not a product behaviour: every documented and scaffolded acceptance command now passes `--clean-alluredir`.
- Part 11 document control not linked to a design input: closed, not a fault. DC-001 is a quality-system document; its links are to the Part 11 clauses it claims, which is correct. A document-to-input link type would add vocabulary for a picture, not for a question anyone asks.

8.2 Findings and actions

- None open from this review.

9 Design Review 8 – Read-only agent interface

Scope reviewed: UN-015, DI-41 (MCP server: schema, query, trace, validate) and DI-42 (read-only), in the graph context.

Disposition: Approved.

9.1 Items reviewed

- MCP over stdio, because agent harnesses (Claude Code, Warp, open-source harnesses) already speak it; the official `mcp` SDK, in the existing `graph` extra rather than a new one.
- Read-only by construction: no write tool, SPARQL Update rejected. An agent that wants a change writes it to the record and opens a pull request, like anyone else.
- Freshness: projected per call rather than from a built store, since projection is cheap; no cache to go stale.
- Kept out: a `propose` tool (that is `git`), HTTP transport, authentication (stdio runs as the local user).
- Fixed in passing: a duplicate `design_inputs` key in the graph design document's frontmatter.
- Addendum: DI-35 now projects each user need's identifier and text; the graph held only the id, so an agent tracing a need could not read it.

9.2 Findings and actions

- None open from this review.

10 Design Review 9 – Risk register

Scope reviewed: UN-016; the new `risk` context with DI-43 (read and score the register) and DI-44 (release-gate rules); DI-45 (risks in the graph, shapes, agent trace); RDM's own tool-risk register (RMF-001).

Disposition: Approved.

10.1 Items reviewed

- A control is a design input. The chain `risk` → design input → test → result needs no new entity, and “every control is verified” is the existing rule that every design input has a passing test.
- Built on RDM-004's settled decision-001: a control changes probability, never severity, so the residual records probability only.
- Kept small against RDM-004's open tickets: one probability per risk (no p1/p2 split, .02), a lookup matrix rather than a formula (.03), and one home for severity and probability — the risk entry (.04). Each can grow later without changing a register written to this format.
- The gate checks only what a machine can: chain, score, control present and declared, residual scored, acceptance named. Substantiating a control in the code and ALARP stay with the reviewer and the risk-analysis skill.
- RMF-001 scores RDM's own tool risks; RISK-TOOL-005 is a Medium residual accepted with its reason, so the acceptance rule is exercised on the record.

10.2 Findings and actions

- Fixed in passing: `rdm story new-input` wrote a second `design_inputs:` key under an empty `[]` (DI-22), and its stub tests failed line-length lint.
- Addendum: DI-34 now runs the test once unmutated first. A test already failing was reported KILLED under any mutation — found while probing DI-44's test before it passed, the false proof the probe exists to rule out.
- Open: halla-health's registers use the cluster-file Markdown format, which the planning-side parser reads; moving them into the record format is work on halla-health, not RDM.

11 Design Review 10 – Risk evaluation per the requirements skill

Scope reviewed: DI-43, DI-44 and DI-45 as amended, measured against the `requirements` skill (scope-impact/agent-skills); RDM's own risk policy (RMP-001) and register (RMF-001).

Disposition: Approved.

11.1 Items reviewed

- No default policy. A matrix's cells are not ISO 14971 values; evaluation needs criteria the project set before the decision, so a register with no declared `risk_policy` blocks ("acceptability criteria missing").
- Acceptability is policy data, not code: per level `acceptable`, `justify` or `unacceptable`. The fixed Low/Medium/High/Block rules are gone; level names are the project's.
- A residual is evaluated only once every control's design input has a passing test; until then it is *not evaluated* and blocks. A passing test shows the control was built as specified; effectiveness remains the reviewer's judgement.
- Safety and security are separate branches: `category`, `stride` for a security risk, `linked` to the safety risk it bears on.
- Residual severity is allowed, defaulting to the initial severity. This departs from RDM-004 decision-001 ("a control never changes severity"), because ISO 14971 lets a control reduce severity and the requirements skill records it; decision-001 should be revisited on that map.
- `status: proposed` marks ratings no person has approved, as a warning. RMP-001 and RMF-001 were written by an agent and are proposed until a maintainer reviews them.

11.2 Findings and actions

- Open: a maintainer to review RMP-001 and RMF-001 and set `status: approved` (or change them).

12 Design Review 11 – One data model: the record and its graph

Scope reviewed: removing DuckDB, the planning tooling and the story-audit context; UN-007 amended; DI-46 (duplicate ids) in the gating context; the gates moved to `rdm/gates/`.

Disposition: Approved.

12.1 Items reviewed

- The record and its RDF graph are RDM's only data model. DuckDB served planning analytics (Backlog and GitHub sync) and SQL in templates, none of it part of the record. Planning lives in its own tools (plan-vs-record).
- Retired: DI-6, DI-13, DI-14, DI-23, DI-32 (see the gating design document). Removed with them: `rdm story audit | sync | backlog-validate | check-ids | validate`, `rdm pm`, `rdm pull`, the template `duckdb` query, and the `story-audit`, `analytics`, `github` and `plan` extras.
- UN-007 keeps its verification: DI-46 makes a duplicated user-need or design-input id fail the design gate. The retired scan read any file for id-shaped strings; DI-46 reads the record, which is what the gates use.
- The gates (`design_gate`, `new_input`, `mutation`) move from the planning package to `rdm/gates/`, so the design gate, release gate, mutation probe and pre-commit hook need no extra.

12.2 Findings and actions

- Open: scope-impact/agent-skills — the `backlog` and `story-audit` skills name removed commands and are updated in the same change.
- Open: projects that used `rdm story sync` or `rdm pm sync` keep their planning data in Backlog.md and GitHub; halla-health's Backlog risk clusters convert to `kind: risk` documents in that repository.

13 Design Review 12 – Findings from the graph's own analysis

Scope reviewed: DI-36 and DI-42 amended; DI-34, DI-37, DI-38 and DI-44 narrowed, with DI-47, DI-48, DI-49 and DI-50 split from them; DI-51 (who landed a change) and DI-52 (no island documents); RISK-TOOL-006 and RISK-TOOL-007.

Disposition: Approved.

13.1 Items reviewed

- Security, found in review: `rdm graph serve` was read-write with open CORS (a cross-origin `CLEAR ALL` emptied the store), and the agent's `query` let `SERVICE` reach the network. Both now refused; recorded as RISK-TOOL-006 and RISK-TOOL-007, which RISK-TOOL-004 had not covered.
- Provenance: the graph showed only authorship, and the latest commit touching a file is never the merge that landed it. DI-51 records the landing commit. It is named *landed*, not *approved*: git cannot see reviewers, and the graph claims only what git records.
- Islands: the V&V plan, the risk policy and the design review connected to nothing. DI-52 links needs, risks and design documents to them. The architecture and matrix template stay unlinked: neither declares anything.
- Requirement size: four design inputs ran past 120 words with one test each. Each is split along its existing clauses so a failing test names the clause.

13.2 Findings and actions

- Open: the reviewer of record lives on the forge (GitHub); projecting it is a later design input.

14 Design Review 13 – Test evidence

Scope reviewed: DI-30 amended (raw results in the release bundle) and DI-53 (steps and attachments in the graph and in `trace`).

Disposition: Approved.

14.1 Items reviewed

- The evidence of record was a status: no test attached anything, results had no steps, and the bundle kept RDM's summary, not the results. Once CI artifacts expired, nothing an auditor could open remained.
- The bundle now retains the results and their attachments. No hashes of its own: the GitHub artifact upload already records a digest.
- Steps and attachments are projected, not judged: no gate requires an attachment, since a test can prove a clause without one and a rule would invite empty attachments. The graph shows which runs carry evidence.
- RDM's own acceptance tests record a step per clause and attach what they checked, starting with the gate and risk tests.

14.2 Findings and actions

- None open from this review.

15 Design Review 14 – Allure results as RDF

Scope reviewed: DI-54 (results in full), DI-55 (fixtures), DI-56 (runs to code) in the graph context; the execution projection moves to its own module, `rdm/graph/allure.py`.

Disposition: Approved.

15.1 Items reviewed

- Allure has no RDF vocabulary. Runs stay `rdm:TestRun` $\subset$ `prov:Activity`, with PROV-O times; labels are kept as name/value pairs rather than one property per label name, so an unknown label needs no vocabulary change.
- Attachment content stays in the files the bundle retains; the graph holds the reference, not the bytes.
- DI-56 closes the code end of the chain the record already carries: the `output` label is on 40 of RDM's 44 runs.
- Split into three design inputs so none outgrows one test.
- Alongside, not a design input: RDM's own acceptance tests record a step per clause, from the clause comments they already carry.
- Addendum: Allure is for acceptance (end-to-end) tests only. They carry the tags, steps and attachments, and each run attaches the text of the design input it verifies; unit tests carry none, held by `tests/allure_scope_test.py`.

15.2 Findings and actions

- None open from this review.

16 Design Review 15 – The Allure behaviors hierarchy

Scope reviewed: DI-31 and DI-40 amended (only `story` names a design input); DI-57 (`rdm.pytest_plugin`), in the verification context.

Disposition: Approved.

16.1 Items reviewed

- Allure's epic → feature → story is RDM's user need → bounded context → design input. Until now both `story` and `feature` named a design input, which left Allure's own hierarchy unusable. `feature` and `epic` now carry the context and the needs; `story` alone is the verification link.
- Derived, not typed: the plugin reads the record when the test runs, so a re-homed input or a new need changes the labels without touching a test.
- The link pins the design document at the tested commit, so the report shows the requirement as it was, not as it is now.
- Opt-in (`pytest_plugins`), so installing RDM changes no one's test run.
- Allure remains for acceptance tests only.
- Addendum: DI-57 links every Markdown document that declares the input, not only its design document — the V&V plan where its user needs are declared and the risk document of each risk it controls — each pinned to the tested commit, so a reviewer reaches the whole record from one result.
- Addendum: browsing RDM's own graph in AWS Graph Explorer showed `explorer-file --exclude TestRun` leaving about a thousand islands — the run details DI-53..56 added (labels, steps, attachments, fixtures, test cases). DI-39 now leaves out, with an excluded class, the nodes that hang only from it.

16.2 Findings and actions

- None open from this review.

17 Design Review 16 – No island documents, second pass

Scope reviewed: DI-58, in the graph context; `contexts`: in the architecture, `references`: in the architecture and the gap-analysis design document.

Disposition: Approved.

17.1 Items reviewed

- Found by browsing the graph, not by a gate: the architecture (SDS-SYS-001), the traceability matrix (TM-001) and document control (DC-001, with its Part 11 clauses) were islands.
- The links come from the record, not from prose: the architecture declares its contexts and their parts in frontmatter; a document lists the documents it relies on in `references`. The matrix is the one exception — it is generated, so its sources are the design documents and the user-need registry, by construction.
- The context list now exists twice, as data and as the descriptive table. The shape catches a design document whose context the data omits; keeping the table's prose in step stays the reviewer's job.
- A reference to a document the record does not hold fails validation: a dangling reference is a broken record, not a style issue.

17.2 Findings and actions

- None open from this review.

18 Design Review 17 – Edges that claim nothing

Scope reviewed: DI-1, DI-22 and DI-52 amended; `satisfies` removed from every design document.

Disposition: Approved.

18.1 Items reviewed

- A critical pass over every edge in the graph found two that state no independent fact.
- `rdm:reviewedIn` linked every design document to the one design review because the design gate requires a review to exist — not because the review covered the document. Removed (DI-52).
- `satisfies` declared, per context, the user needs its design inputs already trace to: the same fact stated twice, and it had drifted (`verification` claimed UN-003, which none of its inputs trace to). A context's needs are now the transitive consequence of design input → `traces_to`; the field is gone from the record, `rdm story new-input` no longer edits it, and the design gate's coverage warning reads the inputs (DI-1, DI-22). An old DHF that still carries `satisfies` is read without error; the key is ignored.

18.2 Findings and actions

- None open from this review.

19 Design Review 18 – Noise out of the graph

Scope reviewed: DI-54 and DI-58 amended; DI-55 retired.

Disposition: Approved.

19.1 Items reviewed

- From the same critical pass over the edges: most of the executions graph repeated the record or said nothing about design controls.
- Labels (565 nodes in RDM's own graph, the largest part of it): `story` and `output` already become `rdm:exercises` and `rdm:exercisesOutput`; `epic` and `feature` are written from the record by DI-57, so projecting them read the record back through Allure; the rest is runner metadata.
- Links (`rdfs:seeAlso`): GitHub URLs repeating the `declaredIn` chain. They stay in the Allure report, where a person clicks them.
- Test cases: one per run while one execution is loaded; nothing to share.
- Fixtures (DI-55, retired): Allure's set-up and tear-down halves became two fixtures each (`tmp_path`, `tmp_path::0`), and `tmp_path` or `capsys` tell a reviewer nothing about a design input.
- The traceability matrix template (TM-001) is output, rendered from what the graph holds; its `prov:wasDerivedFrom` edges (DI-58) were an island-filler. It is no longer projected.
- Nothing is lost as evidence: the raw results, containers and attachments stay in the evidence bundle (DI-30).

19.2 Findings and actions

- None open from this review.

20 Design Review 19 – Evidence that says what it is evidence for

Scope reviewed: DI-59 (verification), DI-60 and DI-61 (graph); DI-35 and DI-51 amended (every controlled document's commit, not only design documents').

Disposition: Approved.

20.1 Items reviewed

- The three gaps the critical pass over the edges left open.
- A run was not tied to the version it tested. The plugin now labels each tagged run with the commit and a dirty working tree (DI-59); the graph links run to commit and warns on an unversioned or stale run (DI-60). A dirty working tree is recorded, not warned on: it is the normal local loop, and CI runs on a clean checkout.

- The claim (source tag) and the evidence (run) never met, and the claim was per file. A test is now a function, defined in its file; a run is a run of that test (DI-61). The two drift warnings apply only once runs link to tests, so a project whose results cannot be matched gets no false alarm.
- Only design documents had a commit and a landing; the V&V plan, the risk documents, the design review and the other controlled documents now do too (DI-35, DI-51). More documents will show as not yet landed until merge.

20.2 Findings and actions

- None open from this review.

21 Design Review 20 – Rules for what the graph derives

Scope reviewed: DI-62, in the graph context.

Disposition: Approved.

21.1 Items reviewed

- Fatemi, Ravanbakhsh and Poole (AAAI 2019, arXiv:1812.03235) removed every triple a background rule implies and found that a model given the rules beat both plain embeddings and rule inference alone, while the plain model, not given them, did worst. The lesson for a slim graph: pruning is safe only for a consumer that knows the rules.
- RDM's derived relations lived in a comment, the docs and the code of `trace`. An agent writing its own SPARQL could not know them. DI-62 makes each rule data in the vocabulary, lists it in the agent server's `schema`, and can materialise it.
- Inferred facts go to their own named graph. In a regulated record a derived link is not evidence; keeping it apart keeps that visible.
- The paper's rules are single-relation subsumptions; `rdm:serves` is a two-step chain, which the paper names as future work. The analogy supports the direction, not the specific rule.
- No learned model enters the record. An agent may use the rules to answer and to suggest; a suggestion becomes part of the record only through a reviewed pull request.

21.2 Findings and actions

- None open from this review.

22 Design Review 21 – One home for each rule and helper

Scope reviewed: a refactor with no change to any design input's text; DI-45's prose updated for how the risk shapes now agree with the gate.

Disposition: Approved.

22.1 Items reviewed

- From a reuse / simplification / efficiency / altitude review of this branch. The risk rules were written twice — in Python for the release gate and again as SHACL — and agreed only because an acceptance test compared them. They are now written once (`rdm/record/risk.py`); the graph carries the gate's findings and the shapes report them. The test that held the two together now also covers a malformed risk policy.
- Git access, namespaces, the gap-analysis matching the graph relies on, and id patterns each get one home instead of several copies.
- A projection reads each document once and asks git for every document's latest commit in one call; the projected graph is unchanged.

22.2 Findings and actions

- None open from this review.

23 Design Review 22 – The gates as reusable CI

Scope reviewed: DI-63, in the scaffolding context.

Disposition: Approved.

23.1 Items reviewed

- A review of the repository's GitHub workflows found the CI `rdm adopt` installs runs `pip install rdm`, which installs a different, older package published under that name: an adopting team's first push goes red.
- DI-63 makes RDM's own CI the reusable unit (a reusable workflow and a composite gates action) and pins it by revision; the adopt template calls it instead of copying steps. RDM's CI calls the same workflow, so the reusable path is the tested path.
- Installing from the pinned revision, not by name, means the gates that ran are the gates that were pinned — the property a regulated pipeline needs.

- The release gate stays switchable: a freshly adopted record has no design input yet, and the gate would fail on that alone.

23.2 Findings and actions

- None open from this review.

24 Design Review 23 – The verification report

Scope reviewed: DI-64, in the verification context.

Disposition: Approved.

24.1 Items reviewed

- The release evidence (DI-30) keeps every Allure result and attachment, but only as JSON and files; the matrix reduces each design input to counts. What a test checked (its clause steps), on which commit, and what it attached, reaches no document a reviewer or an auditor reads.
- DI-64 renders that as a PDF: per design input, every run of its tests with the commit, the worktree state, times, status, failure message, each step's status and every attachment. The header names the results by one SHA-256, so the document is tied to the exact evidence behind it.
- Typst is compiled in-process from the `typst` package, so CI needs no Pandoc; the data goes in as JSON rather than markup, so test output cannot alter the layout. The package is about 77 MB installed, so it is an extra (`report`), not core; without it the bundle says why it has no PDF.

24.2 Findings and actions

- None open from this review.

25 Design Review 24 – The verification report, for an auditor

Scope reviewed: DI-64 (amended) and DI-65, in the verification context.

Disposition: Approved.

25.1 Items reviewed

- The first report, read as an auditor reads a test record (IEC 62304 §5.7 and §9.8, 21 CFR 820.30(f), ISO 14971 §7.2): it did not say who or what ran the tests or in what environment; whether the evidence is fit for a release was spread over 54 runs; failures had to be found by paging; risk controls were not shown; and there was no index and no way to check a file against the retained bundle.
- It also printed what does not help that reader: runner internals (host, thread, framework, language, suite, package), labels repeating the header, the commit and worktree on every run, pages of captured stdout, and the plugin's copy of the requirement text.
- DI-64 is amended to put the reader's questions first — what is this and can I rely on it, what went wrong, what traces to what — then the evidence per design input, then an appendix of file checksums. The left-out data is kept in the bundle; captured output is listed by checksum.
- DI-65 has `rdm.pytest_plugin` record the executor and environment in Allure's own files, so the Allure report shows them as well.

25.2 Findings and actions

- None open from this review.

26 Design Review 25 – One vocabulary with the requirements skill

Scope reviewed: the record's vocabulary (`CONTEXT.md`), the V&V plan's verification approach, and DI-64 (amended).

Disposition: Approved.

26.1 Items reviewed

- RDM said “the test is the acceptance criterion”. The requirements skill keeps acceptance criteria apart from the tests that verify them, and keeps baseline criteria (from the user need) apart from risk-based ones (from a risk control, counted only after verification and an acceptable residual).
- Resolved with the product owner: a design input *is* the acceptance criterion, a system or subsystem *shall* requirement. Baseline when it follows from a user need alone; risk-based when it is allocated as a risk control. The test verifies it, step by step (“verification steps”). The record needs no new structure: the register's `controls`: already allocates risk-based criteria, and the release gate already blocks until controls are verified and residuals acceptable.
- `CONTEXT.md` is rewritten as the glossary, in the domain-modeling skill's format: “risk control” and “control effect” replace “measure” and “measure effect”; a bounded context is the skill's subsystem; “verified” (a passing test) is kept apart from “effective” (verified, and the residual evaluated acceptable); a planning task's acceptance criteria are not the record's.

- DI-64's report followed the old words: it headed test steps “acceptance criteria”, showed “risks controlled” for allocated controls, showed the Allure severity label as if it were a harm's severity, and did not show the risk register's state. It is amended to the glossary.

26.2 Findings and actions

- None open from this review.

27 Design Review 26 – Verification steps are not acceptance criteria

Scope reviewed: the vocabulary (`CONTEXT.md`), the V&V plan's verification approach, the agent workflow.

Disposition: Approved.

27.1 Items reviewed

- Applying the domain-modeling skill to Design Review 25: “clause” meant two things — a standard's requirement in a checklist (`62304:5.2.2`), and a part of a design input that a test step checks — and the test helper named every step `clause(...)`. So a step read as a piece of the acceptance criterion, which the product owner rules out: a test step is not an acceptance criterion.
- Resolved: a design input is one acceptance criterion, accepted or not as a whole; what must be accepted separately is a separate design input (the rule already applied when DI-47..50 were split). A verification step belongs to the test: a named check with its own result, never a criterion or part of one. “Clause” is reserved for a checklist clause.
- Consequences, outside the record: the test helper becomes `verification_step(...)`; the docs, the agent workflow and `rdm story new-input`'s help stop calling the parts of a design input “clauses”. Earlier design reviews keep their wording as the record of their time.

27.2 Findings and actions

- None open from this review.

28 Design Review 27 – The architecture as C4, checked against the code

Scope reviewed: UN-017; DI-66 (record), DI-67, DI-68, DI-69 (graph); the glossary's architecture terms; RDM's own C1, C2 and C3 diagrams; three `realises` entries.

Disposition: Approved.

28.1 Items reviewed

- The product owner's convention: the architecture document holds C1 and C2, each design document its context's C3 (and C4 where needed), as Mermaid C4 in Markdown. The c4-diagrams skill (scope-impact/agent-skills) is the method for drawing them; RDM reads them, never draws them.
- Following the skill: containers are things that run or store data; a component sits in exactly one container; bounded contexts are boundaries around components and may cut across containers (RDM's pytest plugin runs in the acceptance test run, its reusable CI in GitHub Actions); the model is traced to code through `$link`, and to the code's real coupling through its imports rather than through names.
- Conformance findings are warnings: a disagreement between diagram and code is the reviewer's question; neither is assumed right.
- Drawing RDM's own C3 from the evidence (each test run's `output` labels and the import graph) showed three inputs implemented partly in another context: DI-18 (verification) in the gating context's `design_gate.py`, and DI-4 (verification) and DI-61 (graph) in the record context's Allure reader. Recorded as `realises`: gating realises DI-18; record realises DI-4 and DI-61.
- The glossary gains software system, container, component, person, relationship and architecture view; "component" stays a word to avoid for a bounded context itself.
- Mermaid's C4 layout overlaps some relationship labels on the denser views (rendered and inspected); the views stay readable as source, and the model in the graph is what is checked.

28.2 Findings and actions

- None open from this review.

29 Design Review 28 – Mermaid diagrams in rendered documents

Scope reviewed: DI-70 (`design/rendering.md`).

Disposition: Approved.

29.1 Items reviewed

- The C4 views are Mermaid in Markdown. GitHub and the docs site draw them in the reader's browser, but the PDF pipeline (render → Pandoc → Typst) had no step for them, so a PDF printed their source as a code block. Nothing in CI had drawn a Mermaid diagram before: the docs site leaves it to the browser.
- Options weighed: Mermaid's own renderer in a headless browser (exact), a rendering service (sends the record out of the repository: rejected), Graphviz from the parsed C4 model (small,

C4 only, looks different), and browser-free ports (`mermaidx`, `mmdr`: tried on RDM's C4 views, each draws its own layout and one overprints text). Chosen: Mermaid's own renderer, so an auditor's PDF shows the diagram the reviewer saw on GitHub.

- Where it runs: Mermaid's official image, pinned by version and digest, as its own container, not Node and Chrome inside the RDM image. Bundling made every project pull a browser, diagrams or not, and RDM would maintain Chrome's install (an attempt failed on npm 11 skipping install scripts); the official image is maintained upstream for amd64 and arm64. It is large (3.5 GB unpacked, two of its three browsers unused), and only projects with diagrams pull it.
- The step is an RDM Markdown extension, not a Pandoc filter, so it is verified in RDM's test suite (which runs without Pandoc) and runs wherever `rdm render` does. Diagrams are named by a hash of their text, so the render and the Mermaid container agree on each file; the PDF action runs the three steps, and the PDF workflow renders a diagram end to end.
- A diagram that cannot be drawn fails the render: shipping its source in a controlled PDF would pass silently. The renderer runs Mermaid's strict security level; the browser draws only the diagram text from the record.

29.2 Findings and actions

- None open.

30 Design Review 29 – The architecture as one workspace, its views as images

Scope reviewed: DI-66, DI-68 and DI-70 amended; the C4 views of `architecture.md` and every design document; the glossary (`CONTEXT.md`).

Disposition: Approved. Supersedes the Mermaid design of Design Reviews 27 and 28.

30.1 Items reviewed

- The C4 views were Mermaid in each document. Reading them needed a grammar (Mermaid's own, vendored, under Node) and a link workaround, and drawing them in a PDF needed a browser: Mermaid's official image, 3.5 GB. A trial converted RDM's 12 views into one Structurizr workspace: its JSON export carries the whole model (each component's bounded context as its group and its code as a property), and Graphviz draws its DOT export, all 12 views in a third of a second, with no browser.
- The architecture is now one workspace (`dhf/c4/workspace.dsl`), the source; `rdm c4 draw` writes its model as JSON and each view as an image, and each document shows its view as an image. GitHub, the docs site and a PDF show the same picture. The drawn files are committed and stamped with the workspace's hash, and the design gate fails on a stale or missing one, so derived files in git cannot drift from their source. Java and Graphviz are needed to draw, not to check.

- Trade-offs accepted: the record's architecture is more lines than the Mermaid views were (each component's code is a three-line property); a pull request shows the workspace's text and the redrawn image, not an inline diagram; the Graphviz drawings are plainer than Mermaid's.
- Found in the trial and handled in `rdm c4 draw`: Structurizr's DOT export leaves `&` unescaped (it broke 7 of the 12 views); a view that includes `*` leaves out an element related only indirectly (the system context lost the reviewer), so views include what they must show.
- Retired: the Mermaid extension and the PDF action's Mermaid steps (DI-70 as first written); the hand-written Mermaid reader.

30.2 Findings and actions

- None open.

31 Design Review 30 – Bounded contexts drawn by language

Scope reviewed: the bounded contexts of `architecture.md` (ten replaced by eight); every design document; the architecture workspace and its views; the glossary (`CONTEXT.md`). No design input changed: each moved unchanged, with its id, text and the user needs it refines, so its tests, the risks it controls and the verification are unaffected.

Disposition: Approved.

31.1 Items reviewed

- Four analyses of the record agreed: the design inputs clustered by what they do, matched against the contexts and against the user needs; an EventStorming of the domain (events, commands, aggregates); and the imports between components. Six of the ten contexts were named for what their code did (record reads, gating blocks, rendering renders) or for a stage of a design input's life (verification, validation), and ingestion held two design inputs of other contexts' languages. The C4 design inputs sat in four contexts; UN-003 (release blocked) in five.
- The contexts are now drawn where the language changes (`CONTEXT.md`): `specification` (core: DI-1, 2, 5, 15, 22, 24, 26, 31, 33, 40, 46), `release` (DI-3, 4, 18, 30, 63), `test_evidence` (DI-17, 34, 47, 57, 59, 65), `architecture` (DI-66, 68, 70), `compliance` (was `gap_analysis`: DI-10, 11, 12, 25), `publishing` (was `rendering`: DI-7, 8, 9, 16, 29, 64), and `risk` and `graph` as they were, `graph` less DI-68. Retired documents: `record`, `gating`, `scaffolding`, `validation`, `verification`, `ingestion` (SDS-REC, -GATE, -SCAF, -VAL, -VER, -ING-001); new documents SDS-SPEC, -REL, -EVID and -ARCH-001; `compliance` and `publishing` keep their documents' ids.
- `architecture.md` states a dependency rule (a context imports only those below it) and each component's target package. The imports show the code does not keep the rule yet: two cycles, both from the release gate living in the design gate's module and from one Allure

module holding both test tags and results. Moving them is the code change that follows; until then `specification` realises the release gate's parts (DI-3, 18, 44, 50).

- The workspace regroups its components into the eight contexts over today's code paths; the record readers are split, so the Allure reader and the architecture model are components of their own, and `rdm c4 draw` (previously in no component) is one. Ten relationships the code's imports showed were missing are declared.
- Each new design document carries its sources' prose verbatim, marked interim, until it is rewritten for its context.

31.2 Findings and actions

- The interim prose of each design document is to be rewritten for its context.
- The code is to be moved to keep the dependency rule (release gate, Allure reader, frontmatter parser first).

32 Design Review 31 – Every design document to one template

Scope reviewed: the eight design documents; the SDD template (`rdm/adopt_files/dhf/documents/design/example_context.md`); the architecture workspace and its views; the system architecture's dependency rule; the glossary. No design input changed (every document's frontmatter is the one Design Review 30 approved).

Disposition: Approved.

32.1 Items reviewed

- The template, kept agile (what a reviewer needs to approve a change, no padding): Purpose in the context's language; Design Inputs; Design Outputs by component; Components (C3) — the view, a component table, the relationships in words, assumptions and open questions, as the C4 guide asks; Dependencies against the dependency rule; and a dynamic view only where a runtime scenario's order matters and a component view cannot show it. Every design document now follows it; the interim prose of Design Review 30 is gone.
- Each document was rewritten against the code, and where the old prose disagreed, the code won: DI-31 and DI-40 read only `story` tags; the design gate also fails on duplicate ids and stale views; `rdm graph serve` runs the read-only server; the gates action installs `rdm[graph,report]`; the vocabulary extension gives a template the first pass's words rather than expanding anything; DI-68 and DI-69 are stated as not yet built.
- The workspace: component descriptions and relationship labels sharpened; eighteen relationships the code or the pipeline showed were missing are declared; `rdm/first_pass_output.py`, in no component, is one; each component view shows its context's components and their neighbours, without the neighbours' arrows among themselves; three dynamic views (`D_`

`specification_commit`, `D_release_pipeline`, `D_publishing_render`), each step along a declared relationship. Every view was drawn and inspected.

- The glossary's release gate said more than the gate does: it does not require release-grade evidence and only warns on an unvalidated user need. Corrected; Dynamic view added; Effective says what stays a person's judgement.
- The dependency rule names three more imports that break it (the design gate's view check, the plugin's risk labels, the evidence bundle's rendering), each with the change that removes it; the system architecture no longer says DI-68 warns.

32.2 Findings and actions

- Proposed for a later review, not made here: `realises` additions the rewrite surfaced (architecture → DI-67, risk → DI-45, publishing and specification → DI-63, release → DI-64, test_evidence → DI-1, graph → DI-46); amending DI-50 (a risk with an unknown status blocks, as tested) and DI-9 (what the vocabulary extension does).
- Defects found in the code, to be fixed under their design inputs: the ISO 14971 2019 checklist has no clauses (DI-11); an audit against an empty checklist exits 0, and `--coverage` skips a missing checklist path silently (DI-10, DI-12); `rdm translate` misreads an empty `Environment` element (DI-17); the agent's schema omits the architecture and code graphs (DI-41).

33 Design Review 32 – One package per context, and the rule enforced

Scope reviewed: the system architecture's layers and dependency rule; each component's code path in the workspace; the design documents' design outputs, components and dependencies; `realises` in four documents; the glossary. No design input's text changed.

Disposition: Approved.

33.1 Items reviewed

- Each context's code moves into its own package (`rdm/specification/`, `rdm/evidence/`, `rdm/risk/`, `rdm/architecture/`, `rdm/compliance/`, `rdm/release/`, `rdm/publishing/`; `rdm/graph/` as it is), and the shared helpers into `rdm/kernel/`. Three paths users name stay: `rdm/main.py`, `rdm/pytest_plugin.py` and `rdm/md_extensions/`, so no compatibility shim is needed.
- The dependency rule is revised to fit the code and stated as layers in the system architecture's frontmatter: the shared kernel; architecture, risk and compliance (leaves); the specification; test evidence; release; publishing and the knowledge graph. A test reads the layers and the workspace's components and fails on any upward import and on any module no component names (`tests/dependency_rule_test.py`), written first, red on the code as it stood.
- Three splits remove the breaks the rule found: the test tags leave the Allure module for the specification; the release gate, the trace and the design gate's results warnings leave the design gate's module for release (the composition root hands the warnings to the design

gate's output); the frontmatter parser leaves the specification reader for the kernel. The evidence bundle renders, so it is publishing's and realises release's DI-30.

- `realises` follows the code: `specification` keeps DI-61 and DI-70 and no longer realises the release gate's inputs; `release` realises DI-33, DI-44 and DI-50; `publishing` adds DI-30.
- The rewrite runs test-first and keeps the evidence: every acceptance test keeps its tag and its behaviour, so each design input is verified as before; unit tests that cover nothing the others do not are removed, at a line and branch coverage no lower than before (86.1%).

33.2 Findings and actions

- None open.

34 Design Review 33 – A design input is stated once

Scope reviewed: every design document's body; the SDD template; the agent workflow. No design input changed.

Disposition: Approved.

34.1 Items reviewed

- Each design document restated its design inputs in a `## Design Inputs` section beside the frontmatter that declares them: two statements of one requirement, free to drift. The frontmatter is now the only statement. The section is removed from all eight documents and from the template; the body names an input by id where its design outputs say how it is met. Each document's design outputs name every input it owns.
- What the sections held besides the requirement moves where it belongs: the risk register's format to the risk context's design outputs; DI-69's status (not built) to the knowledge graph's. Each input's amendment and split history is in the design reviews that made them.
- The agent workflow says to describe the output, by id, and never to restate the input.

34.2 Findings and actions

- None open.

35 Design Review 34 – The design outputs are the architecture

Scope reviewed: every design document's design outputs; the SDD template; the architecture workspace's component descriptions; the glossary. No design input changed.

Disposition: Approved.

35.1 Items reviewed

- A design output is the architecture that meets a context's design inputs, in the C4 model: its component view, and a dynamic view where the order of interactions matters (`CONTEXT.md`, "Design output"). It names components, never source files or functions: the architecture workspace maps each component to its code. Each design document's outputs are now its component view, a table of each component's responsibility and the design inputs it meets, the relationships in words, and its dynamic views; every input of every document is met by a named component.
- Detail that described the code rather than the design (paths, functions, flags, file layouts, walkthroughs) is cut; what a reviewer needs of it moves into the components' descriptions in the workspace, where the views show it.
- The rewrite found what the code and the record do not yet say alike; each is a question for a later review, not changed here:
 - `realises` that the outputs show but the frontmatter does not declare: `graph` realises part of DI-46; `test_evidence` the Allure ingest of DI-1; `publishing` (the PDF action) and `specification` (adopt's workflow) parts of DI-63; `release` (through its gates action) part of DI-64.
 - DI-30 (the evidence bundle) is owned by `release` and built by `publishing`'s component.
 - DI-44 does not say that a risk with an unknown status blocks, though the code and its test do; DI-59 and DI-65 name a source module in their text.

35.2 Findings and actions

- The `realises`, DI-30 ownership and DI-44, DI-59, DI-65 wording questions above, for a later review.

36 Design Review 35 – Commands and events: the domain model

Scope reviewed: the glossary's new *Domain model* terms; every design document's new *Commands and events*; the SDD template. No design input changed.

Disposition: Approved.

36.1 Items reviewed

- RDM's tactical vocabulary follows the PensionBee DDD workshop (EventStorming, entities, repositories, command handlers, policies): an actor issues a command, resulting in a success event or a fail event that names the business rule it broke, which affects an entity (`CONTEXT.md`, "Domain model"). Two departures, each for a reason:
 - A gate concludes every event its rules produce, not the first, and its verdict is a conclusion over all of them: a reviewer fixes the reasons together, and a warning does not withhold a permission.
 - The workshop's *policy* is RDM's **reaction**: RDM's only policy is the risk policy, and one word must not mean two things in the record.
- The design input has a lifecycle the record now names: declared, approved (committed), verified (a passing tagged test), released; an edit re-opens approval.
- Each design document gains *Commands and events*: its commands, their success and fail events, the entity each affects, and its reactions. The fail events are the rules the code checks today, named; the tools still print prose. Publishing and the knowledge graph hold read models only, which is why they sit in the top layer.
- The EventStorming that produced these found two questions this review leaves open: the change itself (proposed, reviewed, approved, merged) has no context, its approval living in the pull request; and the release decision is recomputed on every run, never recorded as an event.

36.2 Findings and actions

- Name the events in the code, context by context, starting with the release gate: each rule returns a named event with the message printed today (a later change, its own review).
- The two open questions above, for a later review.

37 Design Review 36 – Commands and events, trimmed

Scope reviewed: every design document's *Commands and events*. **Disposition:** Approved.

Each table keeps the commands a reviewer needs and at most four fail events per command, the rules a reviewer thinks in. The full list of rules stays where it is checked: the code and its tests.

38 Design Review 37 – The release gate names its events

Scope reviewed: the release gate's design; the verification data's orphan tags. No design input changed.

Disposition: Approved.

38.1 Items reviewed

- The release gate is the first command to name its events in the code (Design Review 35's action), in the workshop's shape: it fetches the state it needs once, derives every event by its rules, and returns them. Each rule returns a named event (*Release Blocked / Input Untested* and the rest of `release.md`'s *Commands and events*) carrying the message printed today; the verdict, *Release Permitted*, is a conclusion over all of them. What the gate blocks and what it prints are unchanged.
- The rules take the fetched state, never files, so each can be tested without building a DHF.
- The verification data lists only the orphan tags the gates report (those sharing a declared id's prefix): it listed every undeclared tag, so the traceability matrix and the verification report could flag a tag the gates ignore.

39 Design Review 38 – The design gate names its events

Scope reviewed: the design gate's design. No design input changed.

Disposition: Approved.

- The design gate names its events as the release gate does (Design Review 37): each check returns *Design Controls Not Approved / Uncommitted, / Placeholders, / Duplicate Id, / Views Stale* and the rest, each with the message printed today, and each warning a *Design Controls Warned / ...* event; *Design Controls Approved* is the conclusion when none fails. What the gate fails, warns and prints is unchanged.
- One event type serves every context: a name with a slash is a fail event, one with *Warned* before the slash a warning (`CONTEXT.md`, "Domain event").

40 Design Review 39 – The risk rules name their events

Scope reviewed: the risk register's rules. No design input changed.

Disposition: Approved.

- Each risk rule returns a named event with the message it reports today: *Risk Not Evaluated / No Policy, / Incomplete, / Control Unverified, / Residual Unacceptable* and the rest of the rules in the code, and *Risk Warned / Policy Not Approved* and */ Risk Proposed* for the two warnings. A finding stays an event about one risk, or about the register as a whole. What blocks, what warns, and the messages the release gate and the graph's shapes report are unchanged.

- The naming stops at the rules that decide: the design gate, the release gate and the risk register. The other commands' fail events are invocation errors already named in each design document's *Commands and events*; naming them in the code would add no decision.

41 Design Review 40 – The C4 model checked against the record; components in the trace

Scope reviewed: how DI-68 and DI-69 are met. No design input changed.

Disposition: Approved.

- DI-68 (architecture) is realised by the graph's gate shapes, each a warning, never a violation: a design output (a source file a run exercises) in no component's code; a run exercising a component of a context that neither owns nor realises the design input it verifies; a code dependency with no relationship declared from the one component to the other; a component group that is not a bounded context the architecture declares, and a declared bounded context with no component; a context's design document that does not show its component view; a component whose code path does not exist; a relationship with no description. The graph's design document declares it in `realises`.
- Two facts are projected for the shapes, since a shape sees only the graph: a component's code path that does not exist, and the views each design document shows (each `c4/views/<key>.svg` image it holds).
- DI-69: the agent server's trace of a design input names the components whose code its runs exercise, each with its container and context.

42 Design Review 41 – What the C4 shapes found in RDM's own record

Scope reviewed: three design documents' `realises`; three relationships of the architecture workspace. No design input changed.

Disposition: Approved.

DI-68's shapes, run over RDM's own record, warned where the record and the code disagree. Each is resolved in the record, not by quieting a shape:

- The runs verifying DI-31, DI-40 and DI-61 exercise the Allure reader: it holds the one label that names a design input and matches a run to its test by full name. `test_evidence` realises part of each.
- The run verifying DI-63 exercises the PDF action and the workflow `rdm adopt` lays down: `publishing` and `specification` each realise part of it, as Design Review 34 found.

- Three imports had no relationship declared: the new-input command and the verification data on the shared kernel, and the pytest plugin on the test tags. The workspace declares them; its views are redrawn.

43 Design Review 42 – The evidence bundle keeps the run's executor and environment

Scope reviewed: the evidence bundle's contents (DI-30). No design input changed.

Disposition: Approved.

The verification report states the executor and environment the results record and one SHA-256 over every result file, but the bundle kept only the results, containers and attachments: once CI's retention ended, its claims about the run could not be checked. The bundle keeps the two run files the pytest plugin writes (Allure's executor and environment) too. The workspace declares the evidence bundle's use of the Allure reader, whose names for those files it takes; the views are redrawn.

44 Design Review 43 – RDM 2.0.0-alpha, and the version adoption pins

Scope reviewed: how `rdm adopt` stamps the installed version (DI-24, DI-63). No design input changed.

Disposition: Approved.

RDM's next release is 2.0.0-alpha: the record's restructure into bounded contexts, the domain events and the C4 architecture change its module paths. The package version is PEP 440's `2.0.0a0`, but the release is tagged `v2.0.0-alpha` and its image `2.0.0-alpha` (the semantic-version form the image workflow and the PDF action use). Stamped as installed, the workflow `rdm adopt` lays down would pin a tag that does not exist. Adoption stamps the version's release tag instead: a final version unchanged, an alpha, beta or release candidate as `-alpha`, `-beta` or `-rc`, with its number when it is not 0.

45 Design Review 44 – What cannot be read fails the gate

Scope reviewed: the design gate (DI-2) and the release gate (DI-3) over a record or results they cannot read. No design input changed.

Disposition: Approved.

An exploratory test of every RDM command found both gates passing over what they could not read. A frontmatter block was cut at the first `---` anywhere in it, and one that was not YAML read as empty: one unquoted colon, a `---` in a text, or a byte-order mark dropped a design document or the whole risk register, and both gates passed. A result file that did not parse was skipped: a truncated result of a failed run let the release pass.

- Frontmatter runs from a `---` line to the next `---` line (a byte-order mark ignored). The design gate fails, naming the document, when a Markdown document of the DHF has frontmatter that is not YAML, not a mapping, or has no closing fence (event *Unreadable Frontmatter*).
- The release gate blocks when a result file in the results it is given cannot be read as a JSON object (event *Unreadable Result*).

46 Design Review 45 – The risk rules over what a register really says

Scope reviewed: the risk register's rules (DI-44 amended, DI-50).

Disposition: Approved.

The exploratory test found the risk rules passing registers they should block:

- A risk with no controls that declared a lower residual was evaluated at that residual, so a Critical × Likely risk passed. Only a control reduces a risk: with none, the residual is the initial risk (DI-50's own words).
- A risk document whose `risks` was not a list of mappings, or whose kind was written `Risk`, was left out with no finding. DI-44 now blocks on it (*Malformed Register*), and the kind is read in any case.
- A hazard, situation or harm written as a list, a mapping or a number counted as given; only text counts.
- A policy whose severities are numbers was refused as malformed, and a control or link written as one value was read one character at a time.

47 Design Review 46 – Nothing served reaches the network

Scope reviewed: the store, query and endpoint (DI-36 amended), the agent server's refusals (DI-42), RISK-TOOL-007, the SPARQL endpoint container.

Disposition: Approved.

The exploratory test found three ways a read-only query reached the network:

- `rdm graph serve` ran `oxigraph serve-read-only`, which executes `SERVICE` calls. With CORS open, any web page the user has open could make it fetch internal addresses and read the answer. The endpoint is now RDM's own, refusing what the agent server refuses; it reads the store for each query, so it serves the last build and stops with its command.
- The agent server's guard ended a comment at a line feed only; SPARQL also ends one at a carriage return, so `# \rSERVICE <...>` got through. A comment ends at either.
- `rdm graph query` ran `SERVICE` calls too. It refuses them and updates.

RISK-TOOL-007 covers both ways in, and DI-36 is its second control. Two smaller graph faults found by the same test are fixed with them: the sorted N-Quads split a statement at a Unicode line separator in a literal, and the trace of a user need failed on a design input id with a letter part.

48 Design Review 47 – Evidence that says what happened

Scope reviewed: the evidence bundle and report (DI-30, DI-64), the Markdown post-processing (DI-9), result translation (DI-17), the mutation probe (DI-47). No design input changed.

Disposition: Approved.

The exploratory test found evidence that did not say what happened:

- The report and the bundle followed a symbolic link in the results directory, so a link to any file on the machine was printed in the PDF and copied into the bundle as a result. Neither reads a link now.
- The bundle kept only the files the results name, so the report's digest over every result file could not be checked from it; it keeps every plain file. Written over an earlier bundle it mixed the two runs; it replaces them. A missing attachment is listed in the manifest, not dropped, and a report that cannot be laid out leaves the reason, not a crash.
- Audit-note exclusion and section numbering changed code: `[[ -f x ]]` in a shell block was removed and `#` comments numbered. Code is left as written; a note may span lines of a paragraph.
- `rdm translate` reported an errored or skipped xunit test as a pass, and a `qtest` function by its last incident only.
- The mutation probe rewrote a file's line endings while reporting it restored, and a leftover journal overwrote edits made since.

49 Design Review 48 – The rest of what the exploratory test found

Scope reviewed: architecture freshness (DI-70), `rdm story new-input` (DI-22), the pre-commit hook (DI-2), formative usability (DI-5), the record reader (DI-1), gap analysis (DI-10, DI-12). No design input changed.

Disposition: Approved.

- An edit to a file the workspace `!includes` left the views reported current: the digest covered the workspace file alone. It covers its includes.
- `new-input` corrupted a design document whose list was not indented, or added a second `design_inputs` key to a flow list, dropping the context. It reads its edit back and refuses, leaving the document as it was, unless it holds the same inputs plus the new one.
- The pre-commit hook gated eight file types, so a Go, Java, Rust or C change went through a red gate, and no file whose name git quotes. It gates the common languages, and names as written.
- A persona run with an unknown or no outcome counted as completed.
- A user need traced as one value was read one character at a time.
- A stray `[]` made every later mention in the document a reference, and the built-in FDA cybersecurity checklist listed V.A.1.b.ii twice and V.A.1.b.iii never.

50 Design Review 49 – A run finds its test in a nested project

Scope reviewed: the knowledge graph's link from a test run to its test (DI-61). No design input changed.

Disposition: Approved.

- Drawing the worked example's traceability from its graph found none of its eleven runs linked to a test. A test is named by its path from the repository root, and a run by the full name Allure gives it, which starts where pytest runs: the project, the DHF's parent. For RDM the two are the same directory; for a project nested in another repository, such as the example, they are not, so no run found its test and each tagged test was warned as never run. A test is also found by the full name of its path from its project; the test keeps its repository path as its name.

51 Design Review 50 – The record says what it holds, or the gate fails

Scope reviewed: the record reader and the design gate (DI-1, DI-2, DI-46), and `rdm story new-input` (DI-22), against an adversarial behaviour test of the installed package. No design input changed.

Disposition: Approved.

- A design input the reader could not read was dropped without a word, so a requirement nothing tests passed both gates and the graph: `design_inputs` written as a mapping, a string or a list of bare ids; an entry with no `id`, an empty or null one, a list as an `id`, or `ID: for id:: design_inputs` in a document that is not `kind: design`. Each is now a failure of the design gate (*Malformed Declaration*), naming the document and the entry, as is a user need with no `id`.
- A frontmatter that repeats a key kept only the last value: a second `design_inputs` block hid the first's inputs and a duplicate `id`. A repeated key makes the frontmatter unreadable, which fails the gate. So does a document that is not UTF-8, which crashed it.
- A design document or the review counted as approved when git could not see it: ignored, untracked by its rules, a committed link whose target is not committed, or marked to skip or assume-unchanged. Approval now needs the document tracked, clean, and visible to git, and its target too when it is a link.
- Outside git the gate passed saying the design was “approved (committed) in version control”. It still passes, as the design says, but says approval could not be checked.
- `PHOTODOCUMENTATION` read as a placeholder: a placeholder marker is now a whole word, never part of a longer one.
- Two design documents for one context passed, and `new-input` wrote into one of them without saying why. The gate warns, naming both, and `new-input` refuses until one remains.
- Design Review 49 lost this record's *Approval* heading; it is restored.

52 Design Review 51 – A risk rating no one approved is never read as approved

Scope reviewed: the risk register and its release rules (DI-43, DI-44, DI-50) and the risks graph (DI-45), against an adversarial behaviour test of the installed package. No design input changed.

Disposition: Approved.

- A risk or a policy with no status, or an empty one, was read as approved: the gates passed in silence and the graph, the agent's trace and the verification report said *approved*. A missing status is now *proposed*, as DI-43 has it until a person approves the rating, so it warns.
- An acceptance whose `by` or `rationale` was a list, a mapping, a number or `false` counted as who accepted it and why. Both must be text.
- A `kind: risk` document with no `risks` list (a misspelled key, or none) was left out with every risk in it. It now blocks, naming the document, as any other malformed register does; a document that only declares the risk policy is not a register.

- A policy listing a severity or a probability twice gave one pair two levels, and the first won. It is now a malformed policy.
- A recorded residual `level` was not checked against the policy, as a recorded initial level is. It now blocks when it differs.
- A list as a risk id was read as its Python text beside the real id. An id that is not text or a number is now a risk with no id, named as such.
- `rdm graph validate` passed where the release gate blocked on a malformed register or policy with no risk to carry the finding. Such a finding now goes on a stand-in node for the register, so the shapes block it too.

53 Design Review 52 – Every reader of the results agrees with the release gate

Scope reviewed: the Allure reader (DI-4), the release gate and its trace (DI-3, DI-18), the verification data, the verification report and the evidence bundle (DI-30, DI-64), the test tags (DI-40) and the validation records (DI-33), against an adversarial behaviour test of the installed package. No design input changed.

Disposition: Approved.

- A result file the release gate could not read blocked it, while verify, trace, the verification report and the evidence bundle said every input was verified. Each now names the unreadable files: verify exits non-zero, trace lists them, and the report counts them as a reason the evidence is not release-grade.
- Some results that could hold a failed run were read as nothing at all. These are now unreadable: a status Allure does not write (`Failed`, `none`, `null`), labels that are not a list of name and value text, a symbolic link, and JSON nested too deep to decode, which crashed the gate with a traceback.
- A passed test whose verification step failed counted as verified. `CONTEXT.md` says a failed step fails the test, so the reader now treats such a run as failed.
- A run tagged `di-1`, `DI_1` or `DI-1` was dropped without the orphan warning that `DI-9` gets. A tag is now compared on its leading letters, whatever their case or the separator after them.
- The report ignored failed, broken and dirty runs of tests tagged with no declared input, and called the evidence release-grade over a record with uncommitted changes. Every run now counts toward the evidence status, and uncommitted changes to the record are a reason it is not release-grade.
- An evidence bundle written to the folder that holds the results deleted them, then reported release-grade over no files. The bundle now refuses an output that would overlap the results.
- Story decorators on helper functions and classes counted as tags, and `from allure import story` did not. Tags are read only from tests, under any name the file imports the story decorator as.
- Trace said *untested* for a results directory that does not exist, where verify and the gate refuse one; it now refuses too. It also listed the tests of a failed input as “verified by”; they are now “tested by”, each named once.
- An approved validation record with an empty or missing reviewer silenced the warning. A validation needs a person, so it now needs a reviewer.

54 Design Review 53 – The graph blocks what the gates block

Scope reviewed: the projection, the gate shapes, SHACL validation, the agent server and the graph commands (DI-35, DI-36, DI-38, DI-39, DI-42, DI-51), against an adversarial behaviour test of the installed package. No design input changed.

Disposition: Approved.

- `rdm graph validate` passed records the release gate blocks as a whole: a document whose frontmatter cannot be read, a declaration the record reader cannot read, an uncommitted edit, no design review, no design input, and a result file that cannot be read. The projection now asks the release gate for these findings and puts them on the record's node, where a shape reports each one, so `validate` blocks them with the gate's own words.
- The executions graph read results with a reader of its own: it skipped a result with a byte-order mark the gate reads, and a run whose step failed was *passed*. It now uses the Allure reader the gates use.
- The graph used a stricter id grammar than the record: a design input the gates verified, with an id such as `DI-2a`, had no test and no run in the graph, and `trace` refused it. A tag or a story that names a declared id now links, whatever its shape, and `trace` answers any declared id.
- `FROM` and `FROM NAMED` were ignored, so a query read the whole graph while saying it read one. They are refused, naming `GRAPH` instead. `SERVICE` used as a variable or a prefix name is no longer refused.
- A shapes or checklist file that is not valid RDF ended in a traceback with exit 1, the code for a violation. It is an error, exit 2.
- `--infer` with `--store` was ignored, and an unknown `--exclude` class was ignored. Both are refused.
- In a repository with no remote whose one branch is not `main` or `master`, every document was warned as not landed. The default branch is now also git's `init.defaultBranch`, or the only local branch.

55 Design Review 54 – The hook gates every way implementation is committed

Scope reviewed: the pre-commit hook (DI-2) and `rdm hooks` (DI-26), against an adversarial behaviour test of the installed package. DI-26's text changed: it now names the merge hook.

Disposition: Approved.

- Implementation files whose names hold a tab or a double quote committed through a red gate: git quoted the names, and the hook's pattern did not match the quotes. The hook now reads names NUL-separated, exactly as written.
- Upper-case extensions (`app.PY`) and common configuration, template and source types (`config.json`, `settings.ini`, `setup.cfg`, `Dockerfile`, `Makefile`, `index.html`, `App.vue`, `stubs.pyi`,

`analysis.R`, `deploy.ps1`) were not gated. The pattern now matches extensions in any case and includes these.

- A file replaced by a symbolic link was not gated: the hook looked only at added, copied, modified and renamed files. A type change is now gated too.
- A merge brought implementation into a branch whose gate was red, because git runs `pre-merge-commit` for a merge, not `pre-commit`. RDM now ships a `pre-merge-commit` hook that runs the same gate, and installs it with the pre-commit hook (DI-26).
- The hook told a person without RDM to `pip install rdm`, which installs an unrelated package of that name. It now names RDM's own install.

56 Design Review 55 – Gap analysis and publishing say what is wrong

Scope reviewed: gap analysis and the checklists (DI-10, DI-11, DI-12), test result translation (DI-17), rendering and post-processing (DI-7, DI-9), snippets (DI-16) and the DMR index (DI-29), against an adversarial behaviour test and a usability test of the installed package. No design input changed.

Disposition: Approved.

- An audit against a checklist with no clauses (the built-in ISO 14971 2019 holds only its header) said *Success* and exited 0, and coverage skipped it. That was an open question in the compliance design; it is closed: a checklist with no clauses, or one that does not exist, is an error (exit 2) in an audit and in coverage alike, since nothing was checked.
- Coverage exited 0 whatever it found. It now exits 3 when any clause is missing, as the audit does.
- A checklist reached by two spellings of its path (`include ../a.txt`, `include ../a.txt`) was read twice, overstating the clause count. Files are now told apart by their resolved path. An `include` with spaces around the name is read; a bare `include`, a missing file or a directory is an error naming it, not a traceback. A byte-order mark no longer hides the first key.
- A fresh scaffold passes gap analysis, because its templates reference every clause while still holding placeholders. The report now warns, naming each document that still holds a placeholder: its references may describe nothing yet.
- `rdm translate` keyed xunit results by the suite, so three tests named alike in three modules became one row; a test is now keyed by its class name. A Qt5 function skipped with a message disappeared; it is a skip. A failure's text is its message when it has no `message` attribute. A file with no test results in it (an HTML page) is refused, and every translate error is a message with exit 2.
- Audit-note exclusion deleted `[[...]]` text inside a code fence in a blockquote or a list item. Code fences are now found inside containers too.
- A snippet marker was any `RDOC` inside a word (`PRDOC_LIMIT`). Markers are whole words now; a whole-word `ENDRDOC` in another column is still refused, as the design says.
- `rdm story dmr` indexed a blank `id`: as the document `None`. A blank id is no id: the document is skipped with a warning.

- `rdm render` ended in a traceback for an empty configuration, a value the template does not have, or a template that does not exist. An empty configuration is now no configuration (no extensions); the other two are a message with exit 2. Any command whose output is piped into a reader that stops early (`l head`) ends quietly instead of with a traceback.

57 Design Review 56 – Starting a project, adopting one and adding an input work as the docs say

Scope reviewed: `rdm init` (DI-15), `rdm story new-input` (DI-22), `rdm adopt` (DI-24) and the design gate's scope (DI-2), against an adversarial behaviour test and two usability tests (an author and an adopter) of the installed package. DI-24's text changed: it names what adopt now lays down.

Disposition: Approved.

- The design gate held only the design documents and the review. An implementation commit passed while the user needs had uncommitted edits, and the release gate passed with uncommitted risk documents. The documents that declare user needs, and the risk documents (the register and the policy), are record the design rests on: the gate now holds them to the same rule, complete (no placeholder) and committed.
- `rdm adopt` laid down no `.gitignore`, so the acceptance run's own `__pycache__` marked every run dirty, and no render configuration, so the documented matrix render crashed. It now lays down a `.gitignore` of what RDM generates (Allure results, verification data, the graph store, bytecode) and `dhf/config.yml`, each only when absent; when a `.gitignore` exists, the next steps name the lines to add. It wrote through a dangling symbolic link to a file outside the repository; a link, dangling or not, exists, so it is skipped. Its design template linked a C4 view the project does not have; the line is gone, and C4 is named as optional.
- `rdm init` into an existing directory ended in a traceback; it is an error naming the directory (exit 2). On success it printed nothing; it now says what it laid down and the next steps.
- `rdm story new-input` allocated an id that a test was still tagged with, so a retired test verified the new input. An id any test is tagged with is taken. Its stub test file clashed with an existing test module of the same name elsewhere in the suite; it is then named `test_<context>_acceptance.py`. It rewrote CRLF line endings and dropped the comment on an empty `design_inputs: []`; both are kept.
- The project scaffold's Dockerfile needed a wheel in `dist/` that a new project does not have; it now installs the RDM release from git when there is none. Its Makefile did not rebuild Word documents when the reference document changed, and one template fetched an image over the network that the scaffold already holds; both are fixed.
- Gap analysis names a gap as the glossary does (*N gaps*, not *missing items*), and verbose coverage prints no empty heading when nothing is missing.

58 Design Review 57 – Review findings on the behaviour fixes

Scope reviewed: the review of the behaviour fixes' pull request: the design gate's approval check during a merge (DI-2), the record reader's user needs (DI-46), and test result translation (DI-17). No design input changed.

Disposition: Approved.

- The merge hook of Design Review 54 blocked every valid merge that brought an approved design change with its implementation. While a merge is being made, git stages the merged branch's documents, so the design gate read them as uncommitted. A document staged exactly as a commit holds it at its path was committed and reviewed there: during a merge it is approved. (Git names the merged commit only after its merge hook has run, so the commit is found by the document's content.) A merge result no commit holds (a resolved conflict) is new content, and still is not.
- A user need written as a blank string was dropped without a word, where every other entry with no id fails the gate. It now fails it too.
- `rdm translate` accepted a result file whose suites hold no test, and wrote empty data. A file whose results flatten to no test is refused, as one with no suite is.

59 Design Review 58 – Design input by design input, the whole chain

Scope reviewed: each design input in turn, with its user needs, design prose, tagged test, verification steps, output label and last run, read together. Findings are recorded here as each input is reviewed; nothing is changed until the pass is complete.

Disposition: Approved for DI-1 to DI-4, as decided below; the changes are made context by context. The pass continues from DI-5 in a later design review.

59.1 DI-1 – Reading the record, with no planning dependency

Traces to UN-001 and UN-004; described by the Record reader and Test tags rows of the specification context; verified by `test_compile_verification_from_the_record` (output `rdm/specification/sdd.py`), whose last run passed. **Changes requested:**

- Ingesting Allure results is not verified. The test writes a passed result for DI-1 but checks only the total, which counts declared inputs and would be the same were the result ignored. Neither a verified nor an unverified verdict follows from a result.
- The check for no project-management dependency cannot fail: it looks for a module name that no longer exists anywhere. It does not show that reading the record imports no planning tool or needs no `backlog/` directory.

- The text asks for three things to be accepted at once: reading the needs and design inputs, ingesting results, and no planning dependency. Ingesting results is what DI-40 and DI-4 verify. Proposed: narrow DI-1 to reading the needs and design inputs from frontmatter with no project-management dependency.
- The output label names the record reader only.
- One verification step's name is cut off mid-sentence.

Decision (C): narrow DI-1 to reading the user needs and design inputs from frontmatter with no project-management dependency; ingesting Allure results moves to the new reading design input of DI-3's decision (DI-40 reads tags from test source, not result files). Its test is to show that reading the record imports no planning tool and needs no `backlog/` directory, its output label to name what it exercises, and its step names to be whole. Made as listed under *Approved changes*.

59.2 DI-2 – The design gate blocks implementation until the design is approved

Traces to UN-002; described by the Design gate and Pre-commit hook rows of the specification context; verified by `test_design_gate_requires_approval` (output `rdm/specification/design_gate.py`), whose last run passed with fourteen verification steps. **Changes requested:**

- The test verifies more than the text states: the commit and merge hooks, the user-need and risk documents held as the design documents are, the warning on two design documents for one context, and the verdict outside git. Fourteen steps are too many to accept as one input.
- Two steps verify the record reader's handling of declarations and frontmatter it cannot read, which is DI-46's, not the gate's.
- The text names no merge, though the pre-merge-commit hook of Design Reviews 54 and 57 is verified here.
- "Design input" in the text means the design documents; the glossary's design input is a single requirement.
- The output label does not name the hooks (`rdm/specification/hook_files/`).

Decision (C): split. DI-2 stays the design gate over the documents it holds — the design documents, the user needs, the risk documents and the design review — present, complete and committed, re-opened by a later edit, and saying so when approval cannot be checked outside git. A new design input covers the commit and merge hooks that run the gate on implementation work, with its own tagged test and output label. The steps for frontmatter and declarations the record reader cannot read move to DI-46's test. Made as listed under *Approved changes*.

59.3 DI-3 – Release is blocked until every design input is verified

Traces to UN-003; described by the Release gate row of the release context; verified by `test_release_gate_blocks_until_verified` (output `rdm/release/gate.py`), whose last run passed with seven verification steps. **Changes requested:**

- "Every" is not shown: the test's project declares one design input, so a gate that passed when any one input was verified would pass it too.

- A design input with both a passing and a failing run is not tried.
- Three steps verify how Allure results are read — a result that cannot be read, one with a byte-order mark, a status Allure does not write, labels that are not name and value text, a symbolic link, JSON nested too deep — which no design input states. This is the ingestion DI-1 is to drop.
- The step on a run tagged with a mistyped id verifies a warning, not a block, and the text does not state it.
- The output label does not name where results are read (`rdm/evidence/allure.py`).

Decision (C): split. DI-3 keeps blocking release unless every declared design input is verified by a passing test; its test is to declare two inputs, one verified and one not, and to try an input with a passing and a failing run. A new design input of the test-evidence context covers reading Allure results: a result that cannot be read is unverified, never a pass, and a run tagged with an id no design input declares is warned about. It takes the reading steps, the mistyped-id step and its own output label, and the ingestion DI-1 drops. Made as listed under *Approved changes*.

59.4 DI-4 – Results reconcile to a status per design input and render as a matrix

Traces to UN-004; described by the Verification data row of the release context and realised by the Allure reader of the test-evidence context; verified by `test_verification_status_traceable_from_results` (outputs `rdm/evidence/allure.py` and `rdm/release/verify.py`), whose last run passed with four verification steps. **Changes requested:**

- No matrix is rendered: the step named for rendering checks the data the matrix is built from, never the table its template writes.
- “Allure tags” could mean the story labels of the results or the decorators in test source; the test means the labels of the results.
- A skipped result, and a broken one, are not tried: the test does not say which status either gives a design input.
- Two steps verify how results are read — a failed or broken step fails its run, and `verify` names a result file it cannot read — which is the new reading design input’s of DI-3’s decision.
- One verification step’s name is cut off mid-sentence.

Decision (C): reword DI-4 to: “RDM shall reconcile each declared design input against the story labels of the executed Allure results as verified, failed or untested, and render the traceability matrix of those statuses grouped under the user needs.” Its test is to render the matrix and check its rows, to try a skipped and a broken result, and to have whole step names; the two reading steps move to the reading design input. Made as listed under *Approved changes*.

59.5 Approved changes

The decisions above, as made in the record:

- **DI-1** reworded: reading the user needs and design inputs from frontmatter, importing no planning tool and needing no planning directory.
- **DI-2** reworded: the design gate over the design documents, the documents that declare user needs, the risk documents and the design review, and the verdict outside git.

- **DI-71** (specification, new): the commit and merge hooks that run the design gate on implementation work. RISK-TOOL-003 is controlled by DI-2 and DI-71.
- **DI-46** widened to the declarations and frontmatter the record reader cannot read, and the warning on a context with more than one design document, so the steps moved from DI-2's test are stated by the input whose test now holds them.
- **DI-3** unchanged in text; its test to show “every” and a passing and a failing run of one input.
- **DI-4** reworded as decided.
- **DI-72** (test evidence, new): reading Allure results, with what cannot be read never counted as a pass, and orphan tags warned about. It takes the ingestion DI-1 drops.
- The C4 model's component descriptions follow: the hooks, the Allure reader and the verification data.

60 Design Review 59 – Why tests drifted from their design inputs, and the process that holds them

Scope reviewed: how the first four inputs of Design Review 58 came to have tests that verify more, or less, than their text states, and the changes to the way RDM is developed that answer it. No design input changes.

Disposition: Approved.

60.1 What happened

Every design input had a passing tagged test and the gates were green, yet each of the four inputs reviewed disagreed with its test. DI-2's test grew from three verification steps to fourteen in two days of fixes (Design Reviews 44, 48 and 51 to 57) while its text stayed the same: each fix added its proof as a step on the nearest existing test. Other tests verified less than their text: DI-1's ingesting of results, DI-3's “every”, DI-4's rendering.

60.2 Why

- **A fix was recorded but its requirement was not.** Each fix had its design review entry and its amended prose, but whether the behaviour it proved was stated by the input whose test took the step was never asked. The rule that what must be accepted separately is a separate design input existed, and was not applied.
- **No gate reads a test against its text, and none should.** The gates check that a test exists, is tagged and passes; a mutation probe shows a test is sharp, not that it is aimed at its text. The faithfulness gate that read them together was retired (UN-009) because a machine verdict recorded as evidence is weak and approval is a person's; nothing took its place.
- **The review was handed too much.** One pull request carried seven fix batches across every context; no reviewer reads 59 inputs against their tests inside it.
- **Texts joined behaviours with “and”,** inviting a test of one half.

60.3 The answer is a process, not automation

A gate that judged whether a test proves its text would be the faithfulness gate again. The changes are to how RDM is developed, written where the people and agents who change it look:

- **Changing an existing test** (`dhf/AGENT_WORKFLOW.md`, step 5): before a step is added to an existing test, the input's text is read; a step the text does not state means amending the text, or declaring a new input, in the same design review entry.
- **One requirement per input** (`CONTEXT.md`, design input; step 2): one `shall`, no “and” joining behaviours that could be accepted apart.
- **The review reads text and test together** (the pull-request template, step 7): for each design input the change touches, the reviewer reads its text beside its tagged test and says so.
- **One bounded context per fix pull request** (step 7, hard rules).
- **A chain review before each release** (`dhf/AGENT_WORKFLOW.md`, `CONTEXT.md`): every design input read with its user needs, prose, tagged test, verification steps, output labels and last run, recorded as a design review, as Design Review 58 began.

61 Design Review 60 – Design input by design input, continued

Scope reviewed: the chain review of Design Review 58, continued from DI-5: each design input with its user needs, design prose, tagged test, verification steps, output labels and last run. Findings and decisions are recorded here as each input is reviewed; changes are made once approved. From DI-9 on, the pass was kept simple: a change is made only where a test does not check what its text states, where the requirement is asserted outside any verification step, where a step checks what the text does not state (answered by a few words in the text, never by a new input), or where the code passes or drops in silence what it should refuse or report.

Disposition: Approved, with the changes below.

61.1 DI-5 – Persona runs give a formative status per user need

Traces to UN-005; described by the Formative usability and Persona command rows of the specification context; verified by `test_formative_usability_classified` (output `rdm/specification/persona.py`), whose last run passed with one verification step. **Changes requested:**

- Two of the four statuses, clean and not run, are never shown.
- The precedence across a need's runs (failed over issues over clean) is not tried: no need has more than one run.
- `rdm story persona`, which the design says prints each need's status and always passes, is not run.

- A run that names no user need is dropped without a word, and usability issues written as text rather than a list are discarded, so a run that found problems reads as clean. The design says neither; nothing tests either.
- A run naming a user need the registry does not hold is collected but not shown to be reported.
- The test asserts the issues status outside any verification step, and its docstring names UN-001, not UN-005, and claims formative evidence never gates release, which nothing checks.

Decision (C): widen DI-5 so that a persona run file RDM cannot read as a run — no user need, issues that are not a list, JSON it cannot read — is reported and never counted as clean, and a run naming an unknown user need is reported. Its test is to show all four statuses and their precedence over several runs of one need, to run `rdm story persona`, to put every check in a step and to correct its docstring. The reconciler changes with it.

61.2 DI-7 – A Markdown template rendered against the data supplied

Traces to UN-001; described by the Renderer row of the publishing context; verified by `test_renders_template_against_data_context` (output `rdm/publishing/render.py`), whose last run passed with one verification step. **Changes requested:**

- The requirement itself — a template filled from data — is asserted outside any verification step, so the run shows only error handling.
- That one step checks what the text does not state: `rdm render` refuses a value the template uses that the data does not supply, and a template that does not exist, naming each and exiting 2; an empty configuration loads no extensions.
- “A supplied data context” leaves unstated what matters: each data file is available under its file name.
- The test renders a string, not a Markdown template file, except in passing.
- The Renderer row claims rendering in one or two passes, which no design input states and no acceptance test shows; only unit tests do.

Decision (C): reword DI-7 to: “RDM shall render a Markdown template file with Jinja2 against the data files supplied, each available under its file name, and refuse, naming it, a value the template uses that the data does not supply.” Its test is to render a template file from data files in a step, show the file-name convention, and keep the refusals as steps. Two-pass rendering is taken up with DI-9.

61.3 DI-8 – The traceability template filters

Traces to UN-001; described by the Renderer row of the publishing context; verified by `test_traceability_filters` (output `rdm/publishing/render.py`), whose last run passed with three verification steps. **Changes requested:**

- The filters are called as Python functions, never from a template: a renderer that no longer registered them would pass the test.

- `join_to` writes nothing for an id its table lacks, so a document renders a blank row where it should fail (`rdm/publishing/render.py`, `join_to`).

Decision (B, kept simple): DI-8 keeps one input and its text; its test uses each filter from a rendered template, and `join_to` refuses an id its table lacks, naming it.

61.4 DI-9 to DI-70 – the rest of the pass

Approved as they stand: DI-25, DI-35, DI-51, DI-53, DI-54, DI-56, DI-57, DI-58, DI-59, DI-60, DI-62, DI-67, DI-68, DI-69.

Input	Finding	Change
DI-9	Its checks sit outside any step; “expand declared vocabulary” is not what the extension does — a template gets the words its first render produced, which needs the second pass	Text names the second pass; checks in steps
DI-10	The gaps are counted, never listed; exit codes outside a step; steps for nothing checked, a placeholder and a closed pipe unstated	Text widened; the gaps listed, in steps
DI-11	FDA-CYBER and FDA-HFE never checked; includes between built-ins never resolved by name; the ISO 14971:2019 checklist holds no clause	Text asks clauses under unique keys; one step per named standard and one built-in include; the 14971:2019 clauses written
DI-12	The table row outside a step; the unique-keys step is DI-11's	Text: exits as the gap report does; step moved to DI-11
DI-15	Steps check what the text does not state: an existing directory refused, offline build, empty registry, runbook	Text widened
DI-16	Extraction asserted outside a step; a key repeated in a second file silently replaces the first	Step; a repeated key refused, naming both files

Input	Finding	Change
DI-17	gtest asserted outside a step; a file with no results refused, unstated	Text widened; step
DI-18	Unreadable results listed and a missing directory refused, unstated	Text widened
DI-22	Refusals and kept comments and line endings, unstated	Text widened
DI-24	The pre-merge-commit hook never checked; symbolic links and the .gitignore lines unstated	Text widened; the second hook checked
DI-26	No verification steps	Steps
DI-29	Its checks sit outside any step	Step
DI-30	The results-with-attachments check outside a step; steps unstated; a missing matrix template omitted in silence; two steps are DI-64's	Text widened; step; a missing template refused; the two steps moved
DI-31	JavaScript and the other conventional names never exercised; output label names the wrong file	Fixtures added; label <code>rdm/specification/tags.py</code>
DI-33	The named-reviewer rule unstated	Text widened
DI-34	“One-line” is untrue; the ambiguous-site refusal unstated	Text reworded
DI-36	Refusing FROM, <code>--infer</code> over a store and a missing store, unstated	Text widened
DI-37	No verification steps; edition unstated	Text widened; steps
DI-38	Per-rule checks outside any step; whole-record findings and clause shapes unstated	Text widened; steps

Input	Finding	Change
DI-39	A node with its own IRI (a checklist clause from RDF) is left out in silence, and the test asserts it	Vocabulary told apart by the ontology, not the <code>urn:</code> prefix; text widened
DI-40	No story on a test class is tried; output label names the wrong file	Fixture; label <code>rdm/specification/tags.py</code>
DI-41	“Exactly four tools” outside a step	Step
DI-42	Trace accepts a declared id of any shape, unstated	Text widened
DI-43	A malformed policy refused and a status inherited, unstated; a second policy ignored in silence	Text widened; a second policy refused
DI-44	A malformed policy and the policy-only document, unstated	Text widened
DI-45	The projection checked outside a step; register findings and the proposed warning unstated	Text widened; step
DI-47	Restoring is skipped when the file was edited since, unstated	Text widened
DI-48	No verification steps	Steps
DI-49	A missing or non-RDF file reported, unstated	Text widened
DI-50	An unknown status blocks and no status is proposed, unstated	Text widened
DI-52	Both links outside a step; the no-review link unstated	Text widened; step
DI-61	The shape warns only when other tests in the file ran	Text says so
DI-63	The image tag for a branch or commit, RDM’s own CI and the upload, unstated	Text widened; DI-64's workflow sentence moves here

Input	Finding	Change
DI-64	The workflow sentence untested here; several steps unstated	Text widened; the sentence moved to DI-63; the two steps from DI-30 join
DI-65	Unstated labels; <code>worktree=clean</code> written when nothing was checked	Text widened; unknown when the commit is
DI-66	Views, implied relationships unstated; RDM's naming conventions checked as reading; the shared kernel drawn in the specification context	Text widened; the convention check moved to DI-68's own-record step; the kernel drawn in no context
DI-70	An image of no view, removal and refusal, unstated	Text widened

62 Design Review 61 – What a test exercises, without reading the code

Scope reviewed: how the graph learns which components a test run exercises (DI-56, DI-67, DI-68, DI-69), and how to learn it for a project in any language.

Disposition: Approved: proposals 1 to 5.

62.1 Why

A test names the code it exercises in `output` labels, as file paths. Three components of RDM's own model — first-pass output, project templates and the result formatters — read as exercised by no test, though DI-9, DI-15 and DI-17 run their code: each test named the entry point it calls, not the module behind it. The graph says no design input covers them, which is false. Two things make this worse for a project that is not Python:

- the labels are file paths, so they follow the code's layout and break when it moves, and a test claims only what its author remembered to name;
- the dependencies DI-67 and DI-68 compare with the model are Python imports read from the syntax tree (`rdm/architecture/model.py`); a Java or TypeScript project has none.

62.2 Proposed

The C4 model is the language-free description of the code, so the record reasons over it, and reads code only where a project offers it.

1. **A test names components, not files.** DI-56 amended: a test run links to the components its `component` labels name, by the model's component key; a key the model does not declare is a warning, as an unknown design-input tag is. `output` labels keep working, mapped to components by code path as today.
2. **What a named component uses is reached.** A new input, DI-73: the graph derives, by a rule declared in its vocabulary (DI-62), that a run reaches each component the declared relationships lead to from a component it names; reached is kept apart from named, and never stated by the record.
3. **The trace says which.** DI-69 amended: the trace of a design input lists the components its runs name and, apart, those they reach.
4. **Code is read only where a project offers it.** DI-67 and DI-68 amended: the import dependencies stay, for Python only, as a check of the declared relationships; for any other language the declared relationships stand alone, and the record says the check did not run rather than that it found nothing.
5. **Coverage, when a project gives it, is evidence.** A new input, DI-74 (optional for a project): RDM reads per-test coverage in LCOV or Cobertura XML, which `coverage.py`, `JaCoCo`, `Istanbul` and `gcov` write, maps each covered file to its component by code path, and links the run to the components it actually ran, apart from those it names or reaches. RDM parses no source.

62.3 Decisions

- **Only a named component counts for DI-68.** Reaching over-approximates — a component may use another only in paths the test never runs — so a reached component never raises or clears a warning.
- **Coverage (proposal 5) is in this change**, first deferred, then taken in on review. A run's coverage reaches RDM as a file the run attaches to its Allure result, in LCOV or Cobertura XML: any language's Allure adapter can attach a file, so coverage is per test without RDM knowing how it was measured, and RDM still parses no source. Coverage, like reaching, never raises or clears a DI-68 warning: only named components do.

62.4 Approved changes

- **DI-56** amended: a run names components by a `component` label (a model key; an unknown key warns) or, as before, by an `output` label's file.
- **DI-73** (graph, new): the reaches rule, declared in the vocabulary and derived only into the inferred graph.
- **DI-69** amended: the trace lists named and reached components apart.
- **DI-67** amended: Python imports are read for Python code only, and a component whose code holds none is marked as not read.
- **DI-68** amended: the context warning is on named components; the dependency warning is on dependencies read from code.

- **DI-74** (test evidence, new; realised by the graph): a run's coverage attachment links it to the components it ran; an attachment that cannot be read is reported. DI-69 also lists these components, apart.

63 Design Review 62 – RDM's own runs attach their coverage

Scope reviewed: how a test run's coverage reaches the graph for a Python project, RDM's own first among them. No other design input changes.

Disposition: Approved.

- DI-74 reads coverage a run attaches, but nothing attached it: RDM's own graph showed no covered component. **DI-75** (test evidence, new): the pytest plugin, asked with `--rdm-coverage`, measures each tagged test's own coverage with `coverage.py` and attaches it as LCOV to that test's result.
- Coverage is asked for, never assumed: measuring slows a run, and a project may measure coverage its own way. When `coverage.py` is missing, or another measurement (`pytest-cov`, `coverage run`) is already running, the plugin measures nothing and says so rather than disturbing it.
- RDM's own acceptance runs ask for it, so RDM's graph shows what each acceptance test ran. Code a test runs in a separate process is not measured; such a component is still reached or named.

64 Design Review 63 – Coverage is a unit-test measure, never acceptance evidence

Scope reviewed: DI-74 and DI-75 (Design Reviews 61 and 62), against what acceptance evidence is for.

Disposition: Approved.

- Acceptance tests prove design inputs, end to end, and Allure carries their evidence. Code coverage measures how much of a unit's code its tests run: a unit-test measure, which RDM keeps out of Allure and the record as it keeps every unit test out (`tests/allure_scope_test.py`).
- DI-74 and DI-75 measured coverage during acceptance runs and carried it in Allure into the graph. That answers which lines an end-to-end test touched, which no design input asks; it counted the measuring plugin as covered by every run; and it doubled the acceptance run.
- What an acceptance test exercises is answered at the architecture's level, without coverage: the components a run names, and those it reaches through the declared relationships (DI-56, DI-73).
- **Retired:** DI-74 and DI-75, with their tests and code (the coverage reader, `rdm:covers`, the unreadable-coverage warning, the trace's covered components, the plugin's `--rdm-coverage`).

DI-69 returns to named and reached components. The glossary and the workflow's hard rules say that coverage is never acceptance evidence.

- Coverage as evidence of unit verification (IEC 62304 5.5, classes B and C), if wanted, is a design input of its own on the unit-test side, never carried through Allure.

65 Design Review 64 – The unit tests' code coverage, as evidence of unit verification

Scope reviewed: code coverage on the unit-test side, as Design Review 63 left it: evidence that the software units were exercised by their unit tests (IEC 62304 5.5), never acceptance evidence and never carried in Allure.

Disposition: Approved.

- **DI-76** (release, new; realised by test evidence): `rdm story verify`, given the unit tests' coverage as one Cobertura XML or LCOV report (what `coverage.py`, `JaCoCo`, `Istanbul` and `gcov` write), adds to the verification data, per C4 component with code, the lines its unit tests ran out of those measured, and lists apart a component the report does not measure. The traceability matrix shows it beside the design inputs, not inside them: a design input is verified by its acceptance test; a component's unit coverage says how far its own code was unit-tested.
- The C4 model, not a language, maps a file to its component, so any language's coverage report serves. A report that cannot be read is an error, never no coverage.
- No threshold is set and nothing blocks release: a minimum, where a project wants one (for software of class B or C), is a later input.
- RDM's own CI measures its unit tests' coverage and passes it to verify.
- **DI-30** amended: the evidence bundle rebuilt the verification data without the coverage report, so the retained matrix had no coverage section. The bundle takes the same report, carries its coverage as `verify` does, and keeps the report with the evidence.
- A new component, the unit coverage reader (`rdm/evidence/unit_coverage.py`), in the test-evidence context; the views are redrawn.

66 Design Review 65 – The map draws what the graph holds

Scope reviewed: the traceability map of the docs site, drawn from the graph, against what the graph now holds (Design Reviews 61 and 64).

Disposition: Approved.

- The map took the components a run exercises from its output labels only, so it called three components exercised by no test that the graph knows runs name or reach. It draws the

components runs **name** and, apart, those they **reach**, and counts a component exercised when any run names or reaches it.

- The unit tests' code coverage was in the verification data only, so the map, drawn from the graph, could not show it. **DI-67** amended: given the coverage report, the graph carries each component's lines run and measured as the component's own properties — never linked to a run or a design input, so coverage stays unit-test evidence (Design Review 63). `rdm graph build` and the docs build take the report; the map shows each component's unit coverage on its box and in its panel.

67 Design Review 66 – The design outputs follow the behaviour of Design Reviews 58 to 65

Scope reviewed: the design-output tables of the context documents, checked against the graph after PR #54.

Disposition: Approved.

- DI-76 was in no design-output row, and the unit coverage reader component had none: the verification data's row now names DI-76, and the reader has its row (realises DI-76).
- Rows that described behaviour from before the chain review now state it: the renderer's refusals, the snippets' repeated key, the bundle's coverage and refused template, the Explorer file's nodes whatever their IRI, the second risk policy refused, unreadable and orphan persona runs, the worktree unknown without a commit, Python-only import reading. No design input changes.

68 Design Review 67 – The user manual is a design output, held in the DMR and the graph

Scope reviewed: the documentation site, after PR #55 moved its test examples from file paths to component labels and nothing in the record showed which pages still taught file paths.

Disposition: Approved.

- **Why:** a convention changed in prose alone. The graph held the record but not the manual, so no rule, query or gate could say which pages were out of date; they were found by hand.
- **The manual is a controlled document:** `dhf/documents/user_manual.md` (IFU-001, kind: manual) lists its pages, so the DMR index (DI-29) lists the manual. The pages stay in `docs/`: the DMR references them, it does not hold copies. The "RDM on RDM" pages publish this DHF and are not the manual.

- **New user need UN-018:** a user learns from the manual how to keep the record the way the gates enforce, and a reviewer sees which pages a change touches.
- **New design inputs:** DI-77 (graph: the manual's pages, with the design inputs they name and the labels of their test examples, and the pages in a design input's trace), DI-78 (graph: a shape warns on a test example that writes no component label), DI-79 (publishing: RDM's manual shows every test example naming its component).
- **DI-22 amended:** the stub test `new-input` writes carries a component label to fill in, not an output label.
- **C4:** a manual reader component (graph) and a documentation site container holding the user manual and, apart from it, the docs build hooks (`docs/_hooks/`, Python the site build runs), so the manual's code is its pages alone (publishing); views redrawn.
- **The manual is the IFU:** anyone who uses RDM is its user, so the user manual is RDM's instructions for use; `CONTEXT.md` names the term "Instructions for use (IFU)", user manual being its plain name.
- **Not done:** RDM's own acceptance tests keep their output labels (DI-56 allows both); moving them is separate work.

69 Design Review 68 – A blind provenance check, from the graph alone

Scope reviewed: six agents with no context and no file access queried a snapshot of the graph (built at 337693c with fresh evidence), each from one angle: needs forward, evidence backward, documents and git, architecture, risks, and the user manual.

Disposition: Approved.

- **Defect, DI-51:** no document showed a landing commit, although most had landed on `main` through a merge. The landing walk combined `--first-parent` with `--ancestry-path`, which finds the merge only when the document's change is its branch's last commit; in a pull request with later commits it finds nothing. DI-51's text stands; its test gains the case (a change followed by more commits on the branch), and the landing is the oldest first-parent commit of the default branch among the change's descendants.
- **Recorded, not changed here:** every risk is still proposed while six carry a residual decision and one an acceptance (the ratings await a person's approval); a manual page links to a design input only by naming its id, so pages teaching a feature without its id are not found; the DI-79 test names the manual reader, a component of another context that does not realise DI-79; `docs_hooks` is exercised by no test; and no run or commit names a person, the pull-request review being outside the graph.
- **Checked and sound:** every need, input, test and run links forward and back; every run tested the record's commit with a clean worktree; every code dependency has a declared relationship; reached components match the rule exactly; unit coverage is linked to no run or input.

70 Design Review 69 – The instructions for use as one document

Scope reviewed: IFU-001 after Design Review 67: a list of the site's pages in the order the site happened to have, which nobody could read as one document.

Disposition: Approved.

- **The IFU is restructured as instructions for use** (ISO 20417, IEC 82304-1 accompanying documents): a cover with the release and the manual's revision; intended use; how RDM works; safety and limitations; installation; getting started; operating instructions; troubleshooting; reference; revision history (DI-80). The pages move into chapter folders under `docs/`, and IFU-001 lists them in reading order.
- **Residual risks are disclosed** in the safety chapter, each by its register id with what the user must do (DI-81, ISO 14971 disclosure of residual risk).
- **New pages:** the cover, intended use, safety and limitations, operating instructions, troubleshooting, reference. The rest move, unchanged in substance. "RDM on RDM" stays outside the IFU: it publishes this DHF.
- **Paths:** the record's references to moved pages follow them (the DHF overview, the agent workflow, the system architecture, and document control, which goes to revision 2).
- **IFU-001 revision 4:** *Changing the record* comes first in the operating instructions, before the page that details design inputs and tests.
- **Not done here:** rendering the IFU as one PDF.

71 Design Review 70 – The IFU in a developer's words

Scope reviewed: IFU-001 after Design Review 69, read by its users: developers and the people who review their pull requests.

Disposition: Approved.

- **DI-80 amended:** the chapters keep their place and purpose but take the names developer documentation uses: *What RDM is for* (intended use), *How it works*, *Limits and risks*, *Install*, *Quick start*, *Guides* (operating instructions), *Troubleshooting*, *Reference*, *Changelog* (revision history). The cover says it is RDM's documentation, and that it is the controlled IFU-001.
- **Plain words:** the opening chapters drop citations of standards and say what to do instead. Standards stay where RDM checks against them (gap analysis, the risk register, the worked example).
- **Unchanged:** IFU-001 as the controlled document, its reading order, and every residual risk disclosed with what the user must do (DI-81).

72 Design Review 71 – One copy of the docs per release

Scope reviewed: how IFU-001 reaches its readers. One site, rebuilt from `main`, described a release that its own install commands could not give.

Disposition: Approved.

- **New DI-82:** the docs are published once per release tag and once for the development version (`main`), with `mike`; every release stays readable and selectable, and the newest release is the default.
- **DI-80 amended:** the cover's release is filled in by the build: the tag a release build is made from, or the development version. Install commands take the same value, so a reader always installs the release the page describes.
- **Publishing moves to a `gh-pages` branch** (mike's store of versions). The repository's Pages source must be switched to that branch once, by a maintainer.

73 Approval

Recorded in version control (the merged, reviewed PR), per the design-input approval model. No sign-off table is duplicated here.